COURSEBOOK INFORMATICS

As of: 14 October 2026

Online version: https://glandl.github.io/Coursebook-maker-reise-grade-9/en/

COURSEBOOK INFORMATICS

Welcome to the **Maker Journey**: in one school year (2 lessons per week) you will complete five projects covering all competencies of the Informatics curriculum for grades 9–11.

The five projects

1. **Code & Gadget** – Python basics and your first interactive device
2. **Weather Station / IoT** – sensors, networks and data
3. **AI Lab** – from decision trees to generative AI
4. **Data Detectives** – SQL, encryption, privacy
5. **Web Portfolio** – your own project website, developed with agile methods

Along the way you keep a **learning journal** and work in teams using **Kanban and sprints**.

PROJECTS

The five projects of the Maker Journey. Every project follows the same rhythm:

Kickoff & requirements → sprints (build) → review & presentation → retrospective + journal submission

LEARNING JOURNAL

The learning journal accompanies you through the whole year. It is your memory, your evidence and later the material for your web portfolio in P5. It lives as **Markdown files in your own GitHub repository**. Every entry is a commit, so your teacher sees progress continuously, not only at the end of a project.

Repository structure

```
lernjournal/
├── README.md              ← name, class, links to your entries
├── p1/
│   ├── woche-01.md        ← one entry per week
│   ├── woche-02.md
│   └── …
├── p2/ … p5/
├── selbstlernen/
│   └── datenstrukturen.md ← journal tasks from the learning paths
└── images/                ← screenshots and photos
```

How to create the repository and edit files in the browser is explained under Git & GitHub. The syntax is on the Markdown cheat sheet.

What goes in

- **One entry per week** (mandatory): what did I do, what worked, where did I get stuck, what do I plan next?
- **Journal tasks** from the project weeks and the self-study learning paths, each headed with number and title.
- **Evidence:** screenshots, photos of your setup, code snippets, sketches, state diagrams.
- **Sources:** links that helped you, with one sentence on what you took from them.

Template for a weekly entry

File `p1/woche-02.md`:

```
# P1 · Week 2 · 2026-09-21

## Done
- …

## Worked
- …

## Stuck on
- …

## Next step
- …

## Evidence
![Screenshot switch.py](../images/p1-w02-switch.png)

## Sources
- [Thonny guide](…) — helped me find the stop button
```

Rules

1. **Commit by the end of every school week at the latest.** The commit message says what you changed, e.g. `P1 week 2: entry and screenshot`.
2. **Short and honest beats long and pretty.** Three sentences per section are enough.
3. **Evidence goes into the `images/` folder**, not linked from external services.
4. **No other people's data:** no photos of classmates without consent, no passwords, no private addresses. The repository is private, but still.

When is it looked at?

- **Continuously** via the commit history. Your teacher sees when you wrote what.
- Briefly at every **sprint review** (interim presentation) in the team.
- Completely at the **journal submission** at the end of each project. Submission means: the last commit is in, nothing is collected.
- The journal tasks from the self-study learning paths count as evidence that you have worked through the path.

Why this is useful, not annoying

If you keep the journal honestly, you find last time's solution in 30 seconds, you can say concretely in the retrospective what should change, and you already have all the content for the web portfolio in P5. And the commit history shows you yourself how much you got done in a year.

P1 · CODE & GADGET

Duration: ~7 weeks (weeks 1–7) · **Team:** 2–3 people · **Language:** Python (PC) and MicroPython (board)

What is it about?

You learn Python from scratch and end up building an **interactive gadget** on a single-board computer: a device that reacts to inputs (buttons, sensors), moves between different **states**, and answers through outputs (LEDs, display, sound, motor).

Examples of gadgets teams have built or could build:

- Reaction game with two buttons and a high score
- Step counter or dice using the accelerometer
- Night light reacting to brightness and motion
- Timer/Pomodoro clock with display and buzzer
- Alarm system with distance sensor and code entry

The end product

1. A **working gadget** on micro:bit, Raspberry Pi Pico, ESP32 or Arduino (your choice, see MicroPython Boards).
2. A **state diagram** of your gadget (which states exist, what triggers transitions) and a **flowchart** of the main loop.
3. The **source code** with comments and tests, in the team repository: `hardware.py` (board-specific), `logic.py` (testable on the PC), `main.py`, plus `README.md`, `REQUIREMENTS.md` and `TESTS.md`.
4. A **3-minute presentation** at the review in week 7.
5. Your **learning journal entries** (at least one per week).

Learning goals (curriculum)

At the end of P1 you can ...

TB	Competency (curriculum wording, translated)	Grade
02 Algorithms & Programming	Implement algorithms in a text-based programming language using simple applications.	9
04 Systems	Configure computer systems with peripherals, sensors or actuators and basic network functionality and use them for real-life tasks. *(Network part follows in P2)*	10
06 Human-Computer Interaction	Build simple interactive systems and investigate and explain their behaviour by varying inputs and feedback.	10
09 Modelling & Simulation	Abstract and model real objects or situations in a state-based and process-oriented way.	9

Schedule

Week	Phase	Content
1	**Kickoff**	Maker Journey, first Python programs, create the learning journal on GitHub (Markdown, first commit), gadget ideas
2	Kickoff → Sprint 1	Python: variables, branching, loops · choose team & board · requirements for the gadget · set up Kanban board
3	Sprint 1	Functions and lists · bring the board to life: LED, button
4	Sprint 1 → Review	Connect sensors/actuators · draw the state diagram · **Sprint review 1**
5	Sprint 2	Implement the state machine in code · vary behaviour through inputs
6	Sprint 2	Polish, test, document · prepare presentation
7	**Review & Retro**	Presentations · retrospective · journal submission

Tools

- Python & Thonny — from week 1
- MicroPython Boards — from week 3
- Wokwi & Browser Simulators — fallback without hardware
- Git & GitHub and Markdown — from week 1 for the learning journal
- Learning Journal — from day one

Weeks

- Week 1 · Kickoff
- Week 2 · Team, Requirements, Kanban
- Week 3 · Functions, Lists, First Board
- Week 4 · Sensors, State Diagram, Sprint Review
- Week 5 · State Machine and Experiment
- Week 6 · Testing, Code Review, Presentation
- Week 7 · Review and Retrospective {class="children children-type-tree children-sort-weight"}

WEEK 1 · KICKOFF

Today: you get to know the Maker Journey, write your first Python programs, create your learning journal on GitHub and write the first entry in Markdown.

What you will have at the end of the lesson

- Thonny runs on your computer and you have run and modified two programs.
- You know what a **state** is and have simulated a gadget with states.
- You have a GitHub account and a learning journal repository with your first **commit**.

- Your first journal entry is written, in Markdown, with at least two gadget ideas.

1. The Maker Journey (10 min)

One school year, five projects, one learning journal. Have a look at the start page: every project follows the same rhythm **kickoff → sprints → review → retro**. In P1 you build your first interactive device. What it should look like at the end is in the project brief. The boards it will run on are passed around the class today.

2. Tool check: Thonny (15 min)

Open Thonny. If it is missing on your device, follow the setup guide. In an emergency the browser works too: python.microbit.org has a Python console and a simulator.

Type into the **Shell** at the bottom (the window with `>>>`) and press Enter:

```
print("Hello Maker Journey!")
```

If the text appears, you are ready.

3. Two programs (30 min)

Create a new file for each program (**File → New**), save it under the given name and run it with the green play button (or **F5**).

Program 1 · `hello.py` — input and output

```
name = input("What is your name? ")
print("Hello", name, "— welcome to the Maker Journey!")
```

Change it: also ask for a favourite device and print both in one sentence.

Program 2 · `switch.py` — a gadget with two states

Almost every device has **states**: a lamp is OFF or ON. A button press changes the state. That is exactly what we simulate in text here, before we build it with a real LED and button in a few weeks.

```
# A gadget with two states: OFF and ON
state = "OFF"

while True:
    print("Lamp is", state)
    button = input("Press the button [Enter], 'q' to quit: ")
    if button == "q":
        break
    if state == "OFF":
        state = "ON"
    else:
        state = "OFF"

print("Gadget switched off.")
```

Change it:

1. Count how often the button was pressed and print the number at the end.

2. The lamp should switch itself off after the fifth press (“energy saving mode”).

Hint for 1.

Create `presses = 0` before the loop and increase the variable inside the loop with `presses = presses + 1`.

Take a screenshot of the running program. You will need it for the journal in a moment.

If you are fast (or at home) · `trafficlight.py` — three states in a circle

```
state = "RED"
switches = 0

while switches < 6:
    print("Traffic light:", state)
    input("Continue with [Enter]")
    if state == "RED":
        state = "GREEN"
    elif state == "GREEN":
        state = "YELLOW"
    else:
        state = "RED"
    switches = switches + 1

print("Done after", switches, "switches.")
```

Change it: a real traffic light goes RED → RED-YELLOW → GREEN → YELLOW → RED. Add the fourth state. Then draw the states as circles and the transitions as arrows on paper. That is your first **state diagram**.

Preview: the same program on the board

From week 3 your switch runs on real hardware. The core stays the same, only `input` and `print` are replaced by button and LED. You do not have to understand this today, just see that it is the same idea.

micro:bit

```
from microbit import *

state = "OFF"
while True:
    if button_a.was_pressed():
        if state == "OFF":
            state = "ON"
            display.show(Image.HAPPY)
        else:
            state = "OFF"
            display.clear()
```

Raspberry Pi Pico

```
from machine import Pin
import time

led = Pin("LED", Pin.OUT)               # onboard LED
button = Pin(14, Pin.IN, Pin.PULL_UP)   # button between GP14 and GND

state = "OFF"
while True:
    if button.value() == 0:             # pressed
        state = "ON" if state == "OFF" else "OFF"
        led.value(1 if state == "ON" else 0)
        time.sleep(0.3)                 # debounce
```

ESP32

```
from machine import Pin
import time

led = Pin(2, Pin.OUT)                    # onboard LED (usually GPIO 2)
button = Pin(4, Pin.IN, Pin.PULL_UP)     # button between GPIO 4 and GND

state = "OFF"
while True:
    if button.value() == 0:
        state = "ON" if state == "OFF" else "OFF"
        led.value(1 if state == "ON" else 0)
        time.sleep(0.3)
```

Arduino

An **Arduino Nano ESP32** runs MicroPython, so the ESP32 code applies (adjust the LED pin). The **UNO R3** and **UNO R4** cannot run MicroPython: use the Wokwi simulator with a virtual ESP32 or the C++ appendix.

4. Your learning journal on GitHub (25 min)

Your journal is a collection of **Markdown files in a GitHub repository**. You save every change as a **commit**. That way you and your teacher can see it grow week by week.

4a · Account and repository (15 min)

Follow part 1 and part 2 of the guide Git & GitHub. At the end you have a repository `lernjournal` with a `README.md`.

4b · Markdown in five minutes (5 min)

Open the Markdown cheat sheet. Today you only need: `#` for headings, `-` for lists, `**bold**`, and how to embed an image.

4c · First commit (5 min)

Edit the `README.md` in the browser (pencil icon) so that it looks like this, with your details:

```
# Learning journal of <first name>

Class: <class> · School year 2026/27 · Maker Journey

## Entries
- [P1 week 1](p1/woche-01.md)
```

Check the **Preview** tab, then **Commit changes** with the message `README: name and class`. Your first commit.

5. Journal task 1 (15 min)

In the repository create the file `p1/woche-01.md` (**Add file → Create new file**, the slash creates the folder) and write your first entry:

Journal task 1 · Starting point

```
# P1 · Week 1 · <date>
```

```
## Expectations
What do you expect from the Maker Journey? What can you already do
(programming, electronics, tinkering)?

## Gadget ideas
At least two devices from your everyday life you would like to build.
For each idea: inputs (what is measured or pressed), outputs
(what lights up, beeps, moves), states.

## Worked / stuck on
What worked today, where did you get stuck?

## Evidence
Screenshot of switch.py: upload into the images/ folder and
embed it here: ![switch.py](../images/p1-w01-switch.png)
```

Commit message: `P1 week 1: first entry`. If the screenshot upload does not work out today, catch up on it by next week.

If you need inspiration for gadget ideas: this is what a micro:bit can do (German).

▶ *Video: https://www.youtube.com/watch?v=SF_Ym8fWTPU*

Check yourself

What is the difference between `print` and `input`?

`print` outputs text. `input` shows a text, waits for input and returns it as text that you can store in a variable.

What happens in `switch.py` if you delete the line `if button == "q": break`?

The loop runs forever. You can only end the program with the stop button (or Ctrl+C in the shell).

How many states does a bicycle light with the modes off, steady, flashing have?

Three. One button press goes around in a circle: off → steady → flashing → off. That is exactly the structure of `trafficlight.py`.

What is a commit, and why does it need a message?

A commit is a saved state of your repository with time and author. The message says what changed, so that you (and your teacher) can read the history of the journal without opening every file.

Until next week

- Finish journal task 1 and commit it (do not forget the screenshot).
- Try `trafficlight.py` if you did not get to it today.
- Install Thonny at home if you use your own device (guide).
- Video to rewatch if something went too fast today (German):

▶ *Video: https://www.youtube.com/watch?v=oxXAb8IikHM*

WEEK 2 · TEAM, REQUIREMENTS, KANBAN

Today: you learn the three building blocks of every program (variables, branching, loops), find your team, choose your board, write down what your gadget must be able to do, and set up a Kanban board. **Sprint 1** starts today (weeks 2–4).

What you will have at the end of the lesson

- You can convert input into numbers, decide with `if`/`elif`/`else` and repeat with `while` and `for`.
- You are in a team (2–3 people) and you have chosen a board.
- Your team repository contains a file `REQUIREMENTS.md` with user stories.
- Your Kanban board is set up and the cards for sprint 1 are selected.
- Journal task 2 is committed.

1. Look back (5 min)

Show the person next to you your gadget ideas from journal task 1. Which idea has the clearest **inputs**, **outputs** and **states**?

2. Python workshop (30 min)

2a · Variables and data types · `thermostat.py`

A **variable** is a name for a value. The value has a **data type**: `int` (whole number), `float` (decimal number), `str` (text), `bool` (`True`/`False`).

`input()` **always returns text**. You cannot calculate with text, so you convert it: `int(...)` or `float(...)`.

```
# A thermostat decides whether to heat
temperature = float(input("Room temperature in °C: "))
target = 21.0

if temperature < target - 1:
    print("Heating ON")
elif temperature > target + 1:
    print("Heating OFF, open a window?")
else:
    print("Fine. Heating stays as it is.")

print("Difference:", round(temperature - target, 1), "°C")
```

Try it: enter `18`, `21` and `23.5`. Then enter `warm`. Read the error message: which line, which error?

Change it:

1. Ask for the target temperature with `input` as well.
2. Below 5 °C, `Frost protection!` should also appear.

Hint for 2.

The order of the conditions matters: Python takes the **first** branch that matches. So check `temperature < 5` **before** `temperature < target - 1`, or write a separate `if` before the whole block.

2b · Comparing and combining

Expression	Meaning
`a == b`, `a != b`	equal, not equal (a single `=` is an **assignment**)
`a < b`, `a <= b`, `a > b`, `a >= b`	less than, less than or equal, ...
`x and y`	both true
`x or y`	at least one true
`not x`	the opposite

2c · Loops · `countdown.py`

`while` repeats **as long as** a condition holds (you know it from `switch.py`). `for` repeats **a given number** of times or once for each element of a sequence.

```
import time

start = int(input("Count down from: "))

for second in range(start, 0, -1):
    print(second)
    time.sleep(1)

print("Go!")

# Blink pattern: three short, then long
for i in range(3):
    print("*", end=" ")
    time.sleep(0.3)
print("******")
```

`range(start, 0, -1)` counts backwards from `start` to **just before** 0, so down to 1. `range(3)` gives 0, 1, 2.

Change it:

1. For the last three seconds, `Beep!` should appear as well.
2. Keep asking for the start value until a number between 1 and 60 is entered.

Hint for 2.

```
start = 0
while start < 1 or start > 60:
    start = int(input("Count down from (1–60): "))
```

If you are fast · `dice.py`

An electronic dice is a popular gadget. Write a program that prints a random number from 1 to 6 every time you press Enter and says `Again!` on a **six**. After 10 throws it shows how many sixes there were.

```
import random
number = random.randint(1, 6)   # random number from 1 to 6
```

To rewatch (German: branching, then both kinds of loops):

▶ *Video: https://www.youtube.com/watch?v=Kc-JmsKLPjs*

▶ *Video: https://www.youtube.com/watch?v=cSkP6rYQ3U0*

3. Team and board (15 min)

Teams: 2–3 people. Form teams around a shared gadget idea, not just around friendships. Your teacher helps if someone is left over.

Board: all four are equally valid. The choice depends on what your gadget needs.

Board	Strengths	Good for	Watch out
micro:bit (V2)	buttons, 5×5 LED display, speaker, microphone, light, motion and temperature sensor built in. No wiring needed.	quick start, games, step counter, dice	few free connectors, no Wi-Fi
Raspberry Pi Pico (W)	cheap, many pins, analog inputs	night light, alarm system, anything on a breadboard	connect sensors and LEDs yourself. Only the **Pico W** has Wi-Fi.
ESP32	Wi-Fi and Bluetooth built in, fast	anything that should go online later	pinout differs between models
Arduino	robust, widespread	if the board is already there	**UNO R3** and **R4** cannot run MicroPython (the **Nano ESP32** can): then use Wokwi with a virtual ESP32

Looking ahead

In **P2 (weather station)** you send measurements over Wi-Fi. With an **ESP32** or **Pico W** you can keep using your board there. The micro:bit makes P1 easiest, but for P2 you will then need a second board.

Create the team repository: one person creates a **private** repository `p1-gadget-<teamname>` on GitHub (with README), as in Git & GitHub part 2 "create it yourself". Under **Settings → Collaborators** invite the other team members and the teacher. Code, requirements and state diagram live here; your **personal** journal stays in your `lernjournal`.

4. Requirements: what must your gadget be able to do? (25 min)

Before you build, you write down **what** the gadget should do, not **how**. That prevents each team member from building a different device after three weeks.

4a · User stories

A **user story** describes a requirement from the point of view of a person who uses the gadget:

> ***As*** *<role>* ***I want*** *<feature>* ***so that*** *<benefit>.*

Each story comes with **acceptance criteria**: checkable sentences that tell you at the review whether the story is done.

Example night light:

```
### US1 · Light when it is dark (must)
As a child I want the night light to switch on by itself when it gets dark,
so that I do not have to get up in the dark at night.

- [ ] With a hand covering the sensor, the LED switches on within 1 s.
- [ ] In daylight the LED stays off.

### US2 · Switch off (must)
As a parent I want to switch the night light off completely with a button,
so that it uses no power during the day.

- [ ] One button press toggles between OFF and AUTOMATIC.
- [ ] In the OFF state the LED never lights up, not even in the dark.

### US3 · Fade out gently (could)
As a child I want the light to dim slowly so that I do not get startled.
```

Must / should / could says how important a story is. For sprint 1 you only take **must** stories.

How do I recognise a good user story?

- It describes **one** behaviour that can be demonstrated.
- It says nothing about code or pins ("As a user I want a `while` loop" is not a story).
- A stranger can check the acceptance criteria without asking you: "within 1 s" instead of "fast".

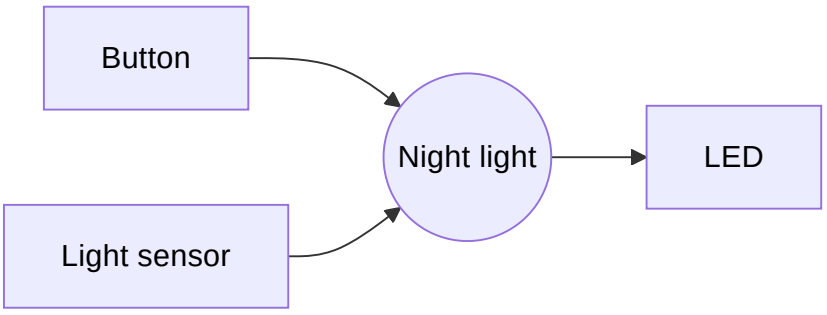
German video on user stories:

▶ *Video: https://www.youtube.com/watch?v=vYnprnrTWwU*

4b · Inputs → gadget → outputs

Add a simple sketch: what goes in, what comes out? GitHub even shows it as a graphic if you write it as a **Mermaid** block in your Markdown file:

````
```mermaid
flowchart LR
    B[Button] --> G((Night light))
    L[Light sensor] --> G
    G --> LED[LED]
```
````



Team task: create `REQUIREMENTS.md` in the team repository with

1. one sentence saying what your gadget is,
2. the sketch inputs → gadget → outputs,
3. **3–5 user stories** with priority and acceptance criteria.

5. Kanban board (10 min)

A **Kanban board** shows at a glance who is working on what and what is done.

Backlog	Sprint 1	In progress	Done
all stories and tasks	what you want to finish by the review in week 4	what someone is doing right now (with name)	acceptance criteria met

Rules:

1. **One card = one task**, small enough for one lesson. Split big stories: “connect LED”, “print sensor value”, “find threshold”.
2. **At most one card per person** in “in progress” (WIP limit). Finish first, then start something new.
3. **Done** means: the acceptance criteria are met, not “almost”.
4. Every lesson starts with a **stand-up** (3 min) in front of the board: What did I do? What am I doing today? Where am I stuck?

You decide where the board lives: **paper/sticky notes** (photo into the repository at the end of the lesson) or digitally as a **GitHub Project** in the team repository (tab **Projects → New project → Board**).

German video on Kanban:

▶ *Video: https://www.youtube.com/watch?v=P513uNckJF0*

6. Journal task 2 (10 min)

Create `p1/woche-02.md` in **your** learning journal and link it in the `README.md`.

Journal task 2 · Team and plan

```
# P1 · Week 2 · <date>

## Team and gadget
Who is in your team? What are you building (one sentence)?
Which board, and why this one and not another?

## My tasks in sprint 1
Which cards from the Kanban board are you taking?

## Python
One thing I understood today (with a code example).
One thing that is still unclear to me.

## Evidence
Photo of the Kanban board or screenshot of thermostat.py with your change.
Link to REQUIREMENTS.md in the team repository.
```

Commit message: `P1 week 2: team, plan, Python`.

Quiz

Quiz · Week 2

1. What happens with this code if you enter `15`?

```
age = input("Age? ")
print(age + 1)
```

a) It prints `16`.

b) It prints `151`.

c) There is an error.

d) It prints `15`.

2. Which numbers does `for i in range(3): print(i)` print?

a) 1, 2, 3

b) 0, 1, 2

c) 0, 1, 2, 3

d) 3, 2, 1

3. `temperature = 22`. What does the program print?

```
if temperature > 25:
    print("hot")
elif temperature > 18:
    print("pleasant")
else:
    print("cold")
```

a) hot

b) pleasant

c) cold

d) pleasant and cold

4. Which one is a good user story?

a) The gadget should be good.

b) As a programmer I want to use a `while` loop.

c) We build a buzzer on pin 15.

d) As a runner I want to see my steps so that I know whether I reached my daily goal.

5. What is the WIP limit (“at most one card per person in progress”) for?

a) So that tasks get finished instead of lots of things lying around half done.

b) So that nobody works too much.

c) So that the teacher has less to check.

d) So that the board looks tidy.

Until next week

- Finish `REQUIREMENTS.md` as a team, fill the Kanban board with cards for sprint 1.
- Commit journal task 2.
- If you have hardware at home: read MicroPython Boards and connect the board to Thonny. Without hardware: open Wokwi or python.microbit.org.

WEEK 3 · FUNCTIONS, LISTS, FIRST BOARD

Today: you package code into **functions**, store many values in **lists**, and bring your board to life: an LED blinks, a button switches it.

What you will have at the end of the lesson

- You can write and call a function with parameters and a return value.
- You can create a list, extend it, loop over it and access elements by index.
- Your board is connected to Thonny, an LED blinks and a button toggles it.
- The board code is in the team repository, the Kanban board is up to date.

1. Stand-up (5 min)

In front of the Kanban board, each person one minute at most: **What did I do? What am I doing today? Where am I stuck?** Move cards, write names on them.

2. Functions (15 min)

A **function** is a named block of code. You write it once with `def` and call it as often as you like. You pass values in as **parameters**, and with `return` the function hands back a result.

`switch_functions.py`

This is `switch.py` from week 1, tidied up: the logic ("what comes after OFF?") and the output are now separate functions.

```
def toggle(state):
    """Returns the new state after a button press."""
    if state == "OFF":
        return "ON"
    return "OFF"


def show(state, presses):
    symbol = "💡" if state == "ON" else "⚫"
    print(symbol, "Lamp is", state, "| button presses:", presses)


state = "OFF"
presses = 0
show(state, presses)

while True:
    button = input("Press [Enter], 'q' to quit: ")
    if button == "q":
        break
    state = toggle(state)
    presses = presses + 1
    show(state, presses)
```

Why the effort? In a few minutes you replace `input` with a real button and `print` with a real LED. The function `toggle` stays **unchanged**. Separate logic can also be tested without hardware (we do that in week 6).

Change it: add a third state `BLINK`: OFF → ON → BLINK → OFF. You only need to change `toggle` (and the symbol in `show`).

3. Lists (15 min)

A **list** stores several values in one variable, in a fixed order.

```
states = ["RED", "RED-YELLOW", "GREEN", "YELLOW"]

print(states[0])        # RED     – index starts at 0
print(states[-1])       # YELLOW  – from the end
print(len(states))      # 4

for s in states:
    print("Traffic light:", s)
```

`traffic_list.py` — states in a circle

```
import time

STATES = ["RED", "RED-YELLOW", "GREEN", "YELLOW"]
DURATION = [3, 1, 3, 1]          # seconds per state

i = 0
for step in range(8):
    print("Traffic light:", STATES[i])
    time.sleep(DURATION[i])
    i = (i + 1) % len(STATES)     # after the last one, back to the first
```

`%` is the **remainder** of a division: `4 % 4` is 0, so the index jumps from the last element back to 0. Names in CAPITALS mean: this value does not change in the program (constant).

`readings.py` — collect and evaluate values

Sensors deliver values continuously. You collect them in a list:

```
readings = []                       # empty list

while True:
    entry = input("Reading (empty = done): ")
    if entry == "":
        break
    readings.append(float(entry))

if len(readings) > 0:
    print("Count:", len(readings))
    print("Smallest:", min(readings), "Largest:", max(readings))
    print("Average:", round(sum(readings) / len(readings), 2))
```

Change it: write a function `average(values)` that returns the average, and use it.

Solution

```
def average(values):
    return sum(values) / len(values)
```

German videos on functions and lists:

▶ *Video: https://www.youtube.com/watch?v=B1mMAieycPY*

▶ *Video: https://www.youtube.com/watch?v=R7lZTvC9UUU*

4. Bring the board to life (45 min)

4a · Connect (10 min)

1. Plug the board in via USB.
2. Thonny: **Tools → Options → Interpreter**, choose the matching MicroPython (for micro:bit: "MicroPython (BBC micro:bit)"). If there is no MicroPython on the board yet, Thonny offers **Install or update MicroPython** at the bottom right.
3. Type `print("Hello board")` into the shell. The answer now comes **from the board**, not from the PC.

Details, drivers and troubleshooting for each board: MicroPython Boards. **No board at hand?** Wokwi (Pico, ESP32) or python.microbit.org (micro:bit with simulator) work with the same code.

⚠ Power off while rewiring

Only connect or move components while the USB cable is **unplugged**. Always use LEDs with a **series resistor** (220 Ω). An LED has a direction: the **long leg** (+) goes towards the pin, the short one to GND.

4b · Blink (10 min)

micro:bit

No wiring needed. The LED in the middle of the 5×5 display has the coordinates (2, 2).

```
from microbit import *

for i in range(10):
    display.set_pixel(2, 2, 9)   # brightness 0–9
    sleep(500)                   # milliseconds
    display.set_pixel(2, 2, 0)
    sleep(500)
```

Raspberry Pi Pico

First the **onboard LED**, no wiring needed:

```
from machine import Pin
import time

led = Pin("LED", Pin.OUT)

for i in range(10):
    led.value(not led.value())   # toggle
    time.sleep(0.5)
```

Then an **external LED**: GP15 → 220 Ω → long LED leg, short leg → GND. In the code replace `Pin("LED", …)` with `Pin(15, Pin.OUT)`.

ESP32

First the **onboard LED** (GPIO 2 on most DevKit boards):

```
from machine import Pin
import time

led = Pin(2, Pin.OUT)

for i in range(10):
    led.value(not led.value())   # toggle
    time.sleep(0.5)
```

No onboard LED, or it stays dark? External LED: GPIO 18 → 220 Ω → long LED leg, short leg → GND, in the code `Pin(18, Pin.OUT)`.

Arduino

Nano ESP32: code as in the ESP32 tab, adjust pin numbers according to your board's pinout on MicroPython Boards. **UNO R3 / R4** (no MicroPython): create an ESP32 project in Wokwi and use the ESP32 code.

Change it: make the LED blink three times short and three times long.

4c · Button toggles LED (15 min)

Now `toggle` from part 2 moves onto the board, **unchanged**.

micro:bit

```
from microbit import *


def toggle(state):
    if state == "OFF":
        return "ON"
    return "OFF"


def show(state):
    if state == "ON":
        display.show(Image.HEART)
    else:
        display.clear()


state = "OFF"
show(state)

while True:
    if button_a.was_pressed():   # True only once per press
        state = toggle(state)
        show(state)
    sleep(20)
```

Raspberry Pi Pico

Wiring: button between **GP14** and **GND**. External LED on GP15 as in 4b (or onboard LED `Pin("LED", Pin.OUT)`).

```
from machine import Pin
import time

led = Pin(15, Pin.OUT)
button = Pin(14, Pin.IN, Pin.PULL_UP)   # not pressed = 1, pressed = 0
before = 1
```

```
def pressed():
    """True only at the moment the button goes down."""
    global before
    now = button.value()
    new = before == 1 and now == 0
    before = now
    return new


def toggle(state):
    if state == "OFF":
        return "ON"
    return "OFF"


def show(state):
    led.value(1 if state == "ON" else 0)


state = "OFF"
show(state)

while True:
    if pressed():
        state = toggle(state)
        show(state)
    time.sleep_ms(20)                   # also debounces
```

ESP32

Wiring: button between **GPIO 4** and **GND**. LED: onboard on GPIO 2 or external on GPIO 18.

```
from machine import Pin
import time

led = Pin(2, Pin.OUT)
button = Pin(4, Pin.IN, Pin.PULL_UP)    # not pressed = 1, pressed = 0
before = 1


def pressed():
    """True only at the moment the button goes down."""
    global before
    now = button.value()
    new = before == 1 and now == 0
    before = now
    return new


def toggle(state):
    if state == "OFF":
        return "ON"
    return "OFF"


def show(state):
    led.value(1 if state == "ON" else 0)


state = "OFF"
show(state)

while True:
    if pressed():
        state = toggle(state)
        show(state)
    time.sleep_ms(20)                   # also debounces
```

Arduino

Code as in the ESP32 tab, adjust the pins (see MicroPython Boards). UNO R3 / R4: Wokwi with ESP32.

Why `pressed()` and not simply `button.value() == 0`?

The loop runs 50 times per second. A button press lasts about 0.2 s, so `button.value() == 0` would be true ten times in a row and toggle the LED ten times. `pressed()` remembers the last value in `before` and only reports the **change** from "up" (1) to "down" (0). This is called **edge detection**. `global before` allows the function to change the variable outside of it. On the micro:bit, `was_pressed()` does this for you.

Change it: bring over your third state `BLINK` from part 2. Hint: in the BLINK state, toggle the LED in every loop pass if 300 ms have passed since the last toggle (`time.ticks_ms()`, on the micro:bit `running_time()`).

4d · Start without a PC (5 min)

A gadget should also run without a laptop. In Thonny save the program via **File → Save as → MicroPython device** as `main.py`. The next time you plug it in (even into a power bank) it starts automatically. On the micro:bit: **Flash/Send to micro:bit** in the editor.

Videos to rewatch, in German (Pico: external LED and reading a button, ESP32: first steps with Thonny):

▶ *Video: https://www.youtube.com/watch?v=DaZF-3jM69U*

▶ *Video: https://www.youtube.com/watch?v=HTIlg6IqNzA*

▶ *Video: https://www.youtube.com/watch?v=59S53NSHR0M*

5. Into the team repository (5 min)

Upload your board code to the team repository (**Add file → Upload files**), for example as `code/main.py`. Update the Kanban board: what is **done**?

6. Journal task 3 (10 min)

Journal task 3 · First sign of life

```
# P1 · Week 3 · <date>

## Setup
Photo of your board with LED and button (or a screenshot from the simulator).
Which pins did you use?

## A function explained
Copy a function from your code here (as a ```python block)
and explain in 2–3 sentences: which parameters does it take,
what does it return, why is it a function of its own?

## Problem and solution
What did not work straight away, and how did you solve it?
```

Commit message: `P1 week 3: board is running`.

Quiz

Quiz · Week 3

1. What does this program print?

```python
def double(x):
    return 2 * x
```

```
print(double(3) + 1)
```

a) `8`

b) `7`

c) `2 * 3 + 1`

d) Nothing, because `double` does not print anything.

2. `colours = ["red", "green", "blue"]`. What is `colours[1]`?

a) `"red"`

b) `"blue"`

c) `"green"`

d) an error

3. `i = 3` and the list has 4 elements. What is `(i + 1) % 4`?

a) 0

b) 4

c) 1

d) 3

4. A button is connected between a pin and GND, and the pin is configured with `Pin.PULL_UP`. What does `button.value()` return while the button is pressed?

a) 1

b) `True`

c) It depends on the button.

d) 0

5. Why do we keep `toggle(state)` separate from `show(state)`?

a) Because Python requires it.

b) So that the logic stays the same on PC and board and can be tested without hardware.

c) So that the program runs faster.

d) Because a function may have at most three lines.

Until next week

- Commit journal task 3.
- As a team: which **sensor** and which **actuator** (buzzer, servo, display, ...) belong to your gadget? Reserve the part with your teacher.
- Next week is **sprint review 1**: your board should be able to demonstrate at least one must story by then.

WEEK 4 · SENSORS, STATE DIAGRAM, SPRINT REVIEW

Today: your gadget gets senses (sensor) and a voice (actuator). You draw which **states** it can be in, and you show your progress at **sprint review 1**.

What you will have at the end of the lesson

- Your board has a file `hardware.py`: everything that depends on the board is inside it.
- You have watched sensor values live in the plotter and chosen a threshold.
- Your team has a **state diagram** and a **flowchart** of the gadget.
- You demonstrated at the sprint review, got feedback and adjusted the backlog for sprint 2.

1. Stand-up (5 min)

As always in front of the board. Additionally: **What do we show at the review today?** One person prepares the demo.

2. The hardware layer · `hardware.py` (15 min)

Last week every program contained board-specific code (`Pin(14, …)`, `display.show(…)`). From today you separate this cleanly:

- `hardware.py` contains small functions such as `brightness()`, `led(on)`, `tone(…)`, `pressed()`. Only this file differs from board to board.
- `main.py` and all other programs only use these functions. They are **the same for all boards**.

In Thonny save the matching file as `hardware.py` **onto the board** (File → Save as → MicroPython device). You only need the functions your gadget uses. You add a different sensor (distance, motion, temperature) the same way: one function that returns a number. Wiring diagrams for more components: MicroPython Boards.

micro:bit

Wiring: none, everything is built in (micro:bit **V2** for sound). The LED display also measures brightness. If it is lit at that moment, the measurement is slightly off.

```
# hardware.py – micro:bit V2
from microbit import *
import music


def brightness():
    """Ambient light in percent (0 = dark, 100 = bright)."""
    return display.read_light_level() * 100 // 255


def tilt():
    """Tilt to the left/right, roughly -100 to 100."""
    return accelerometer.get_x() // 10


def led(on):
    if on:
```

```
        display.show(Image.HEART)
    else:
        display.clear()


def tone(frequency, duration_ms):
    music.pitch(frequency, duration_ms)


def pressed():
    """True once per press of button A."""
    return button_a.was_pressed()
```

Raspberry Pi Pico

Wiring (USB unplugged):

Component	Connection
Button	GP14 ↔ GND
LED + 220 Ω	GP15 → resistor → LED (long leg) → GND
Passive buzzer (piezo)	GP16 ↔ GND
Light sensor (LDR) + 10 kΩ	3V3 → LDR → **GP26** → 10 kΩ → GND

Instead of the light sensor you can also use a **potentiometer** (outer legs to 3V3 and GND, middle to GP26).

```
# hardware.py — Raspberry Pi Pico
from machine import Pin, ADC, PWM
import time

_button = Pin(14, Pin.IN, Pin.PULL_UP)
_led = Pin(15, Pin.OUT)
_buzzer = PWM(Pin(16))
_buzzer.freq(1000)
_buzzer.duty_u16(0)
_sensor = ADC(26)
_before = 1


def brightness():
    """Ambient light in percent (0 = dark, 100 = bright)."""
    return _sensor.read_u16() * 100 // 65535


def led(on):
    _led.value(1 if on else 0)


def tone(frequency, duration_ms):
    _buzzer.freq(frequency)
    _buzzer.duty_u16(32768)            # on for half the period = loudest tone
    time.sleep_ms(duration_ms)
    _buzzer.duty_u16(0)


def pressed():
    """True once per press (edge detection)."""
    global _before
    now = _button.value()
    new = _before == 1 and now == 0
    _before = now
    return new
```

ESP32

Wiring (USB unplugged):

Component	Connection
Button	GPIO 4 ↔ GND
LED + 220 Ω	GPIO 18 → resistor → LED (long leg) → GND (or onboard LED GPIO 2)
Passive buzzer (piezo)	GPIO 19 ↔ GND
Light sensor (LDR) + 10 kΩ	3V3 → LDR → **GPIO 34** → 10 kΩ → GND

Instead of the light sensor you can also use a **potentiometer** (outer legs to 3V3 and GND, middle to GPIO 34).

```
# hardware.py – ESP32
from machine import Pin, ADC, PWM
import time

_button = Pin(4, Pin.IN, Pin.PULL_UP)
_led = Pin(18, Pin.OUT)
_buzzer = PWM(Pin(19))
_buzzer.freq(1000)
_buzzer.duty_u16(0)
_sensor = ADC(Pin(34))
_sensor.atten(ADC.ATTN_11DB)           # full range 0–3.3 V
_before = 1


def brightness():
    """Ambient light in percent (0 = dark, 100 = bright)."""
    return _sensor.read_u16() * 100 // 65535


def led(on):
    _led.value(1 if on else 0)


def tone(frequency, duration_ms):
    _buzzer.freq(frequency)
    _buzzer.duty_u16(32768)
    time.sleep_ms(duration_ms)
    _buzzer.duty_u16(0)


def pressed():
    """True once per press (edge detection)."""
    global _before
    now = _button.value()
    new = _before == 1 and now == 0
    _before = now
    return new
```

Arduino

Nano ESP32: use the ESP32 file and adjust the pin numbers according to the pinout on MicroPython Boards. Keep the function names, then the rest of the code runs unchanged. **UNO R3 / R4** (no MicroPython): Wokwi with ESP32 and the same wiring.

The underscore in front of `_button`, `_led` ... means: only for use **inside** `hardware.py`.

3. Explore the sensor · `sensor_test.py` (10 min)

This program is the same for all boards:

```
from hardware import *
import time
```

```
while True:
    print(brightness())
    time.sleep_ms(200)
```

Run it and open **View → Plotter** in Thonny. Cover the sensor with your hand, shine your phone light on it.

Decide as a team: which value is "bright" (daylight in the room), which is "dark" (hand over it)? Choose a **threshold** in between and write it down.

4. Sensor controls actuator · `nightlight_simple.py` (10 min)

```
from hardware import *
import time

THRESHOLD = 30          # your value from part 3


def is_dark(value):
    return value < THRESHOLD


while True:
    led(is_dark(brightness()))
    time.sleep_ms(100)
```

Try it and observe:

1. Slowly move your hand over the sensor, exactly at the border. What does the LED do?
2. How would you switch the night light **off completely** when you do not need it?

This program cannot solve question 2: it knows no **states**. It always does the same thing, no matter what happened before. For that you need a model.

<details>
<summary>What happens in 1.?</summary>

The LED flickers. The reading fluctuates around the threshold, and every small change switches it. Next week you look at how to prevent that.
</details>

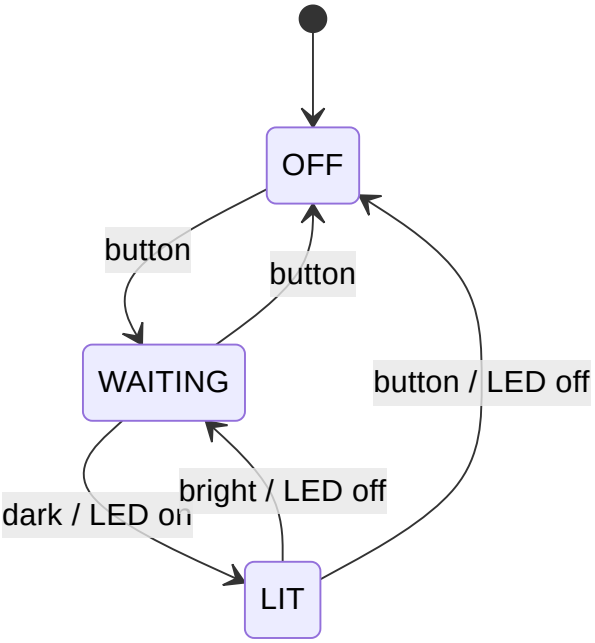
5. State diagram and flowchart (15 min)

5a · State diagram: *which state is the device in?*

A **state diagram** shows

- **states** (rounded boxes): situations in which the device behaves the same way,
- **transitions** (arrows): *event / action*, e.g. `button / LED off`,
- the **initial state** (arrow from the black dot).

The night light with an off button:



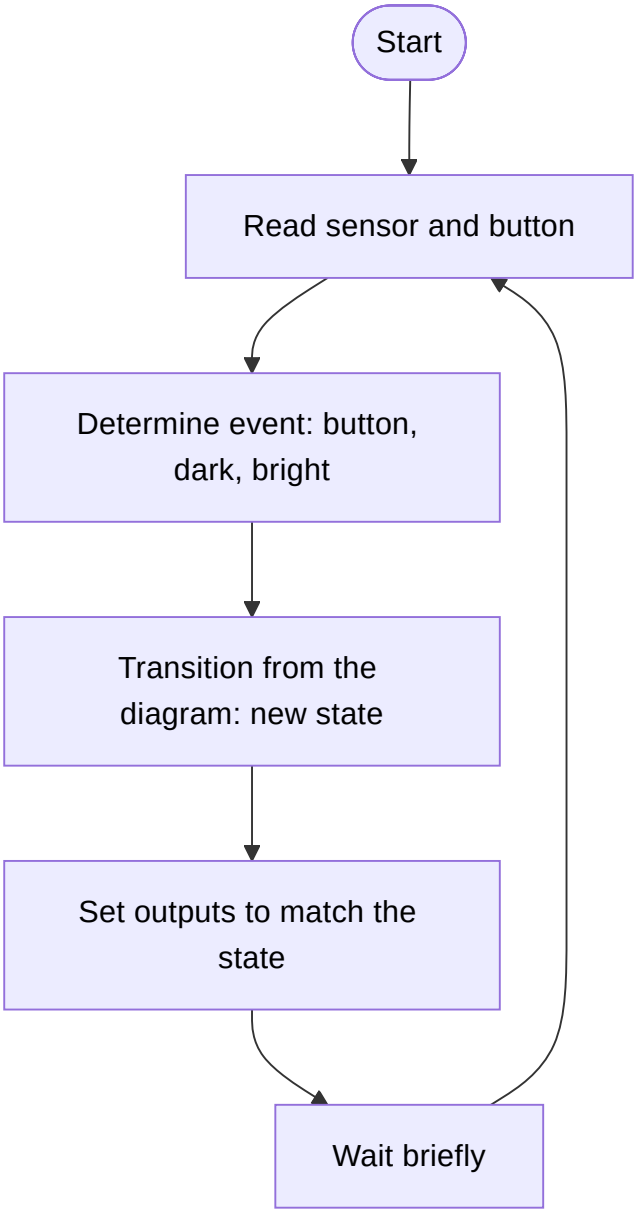
Check the diagram: is there an answer for every event in every state? What happens in state OFF when it gets dark? (Nothing. A missing arrow means: the state stays.)

German video: state diagram of a ticket machine.

▶ *Video: https://www.youtube.com/watch?v=nwmlg0ZYnF8*

5b · Flowchart: *what does the program do, step by step?*

The state diagram says **what** the device does. The **flowchart** shows **how** the main loop implements it. You need both: one models the device, the other models the program.



Team task: draw the state diagram of **your** gadget, on paper first. Put it into the team repository as `STATE_DIAGRAM.md`, either as a photo or as a Mermaid block (GitHub shows it as a graphic):

````
```mermaid
stateDiagram-v2
    [*] --> OFF
    OFF --> WAITING: button
```
````

State diagram checklist

- Each state has a name that describes a **situation** (WAITING, LIT), not an activity of the program (READ_SENSOR).
- Exactly one initial state.
- Every arrow is labelled with an **event**.
- You can leave every state again (no dead end, unless that is intended).
- A person who does not know your device can predict from the diagram what a button press does.

6. Sprint review 1 (35 min)

The **sprint review** is an interim presentation: you show what **works**, not what you are planning. The audience is your classmates; they give feedback like customers.

4 minutes per team:

Time	Content
2 min	**Demo** on the board: which user stories are done? Read the acceptance criteria aloud and demonstrate them.
1 min	Show the **state diagram**: this is how the finished gadget should work.
1 min	**Feedback** from the class

Giving feedback (short, friendly, specific):

- *I like ...*
- *I wonder ...*
- *What if ...*

After all demos (5 min in the team): write the feedback on new cards, re-sort the backlog, choose the cards for **sprint 2** (weeks 5–6). Unfinished items from sprint 1 go back into the backlog, not automatically into sprint 2.

7. Journal task 4 (5 min, finish at home)

Journal task 4 · Model and review

```
# P1 · Week 4 · <date>

## Our model
State diagram of your gadget (photo or Mermaid block).
Explain one transition in your own words:
"In state … , … causes …"

## Sensor
Which sensor, which threshold, how did you find it?

## Feedback from the review
The most important feedback you got,
and what you are changing in sprint 2 because of it.
```

Commit message: `P1 week 4: state diagram and review`.

Quiz

Quiz · Week 4

1. Why is the board-specific code in `hardware.py`?

a) Because MicroPython only allows two files.

b) So that the rest of the code stays the same on all boards, and you can switch boards without rewriting everything.

c) So that the program starts faster.

d) Because `main.py` must not contain imports.

2. Which of these names is a good **state** for a state diagram?

a) `READ_SENSOR`

b) `WHILE_LOOP`

c) `ALARM_ACTIVE`

d) `SWITCH_LED_ON`

3. In the diagram above: the night light is in state **OFF** and it gets dark. What happens?

a) Nothing, it stays in state OFF.

b) It changes to LIT.

c) It changes to WAITING.

d) The diagram is wrong.

4. What is the difference between a state diagram and a flowchart?

a) There is none, they are two names for the same thing.

b) The flowchart shows the states, the state diagram the code.

c) The state diagram is for Python, the flowchart for MicroPython.

d) The state diagram models how the device behaves. The flowchart shows in which order the program works.

5. What belongs in the demo at a sprint review?

a) Everything you still want to build until the end of the project.

b) What already works, measured against the acceptance criteria.

c) The entire source code, line by line.

d) Only slides, so that nothing can go wrong.

Until next week

- Commit journal task 4.
- Finish `STATE_DIAGRAM.md` in the team repository: next week it turns into code.
- If you like, a German video on how a sensor works and how to program it (shown on the micro:bit, applies to all boards):

▶ *Video: https://www.youtube.com/watch?v=rqTMaD4FnFA*

WEEK 5 · STATE MACHINE AND EXPERIMENT

Today: sprint 2 begins. You translate a state diagram into code using a fixed recipe, build a reaction game, and investigate in an **experiment** how the behaviour of an interactive system changes when you vary inputs and feedback.

What you will have at the end of the lesson

- You can implement a state diagram in Python using the **state machine recipe**.
- The night light with an off button runs (simulated on the PC and/or on the board).
- You have measured reaction times under different conditions and explained the result.
- Your own gadget has a state machine following the recipe, at least for some of its states.

1. Stand-up (5 min)

Sprint 2 has started: which cards did you choose after the review? Who takes which?

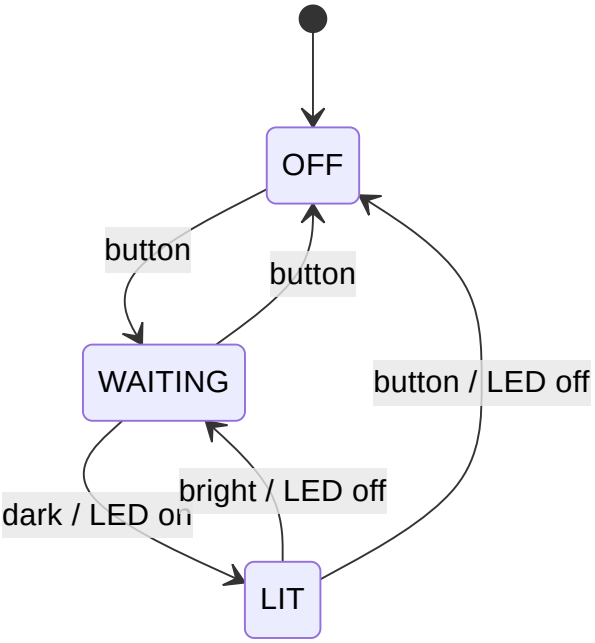
2. From diagram to code (30 min)

2a · The state machine recipe

Every gadget with states can have the same structure:

1. **One** variable `state` holds the current state.
2. A function `next_state(state, event)` contains **exactly the arrows** of the diagram: one `if` block per state, inside it one `if` per outgoing arrow. No arrow matches? The state stays.
3. The main loop: **determine event → compute new state → on a change, set the outputs**.

As a reminder, the diagram from week 4:



2b · Simulate on the PC · `nightlight_pc.py`

You type the events instead of measuring them. That way you test the logic without hardware.

```
def next_state(state, event):
    """The arrows of the state diagram, nothing else."""
    if state == "OFF":
        if event == "button":
            return "WAITING"
    elif state == "WAITING":
        if event == "button":
            return "OFF"
        if event == "dark":
            return "LIT"
    elif state == "LIT":
        if event == "button":
            return "OFF"
        if event == "bright":
            return "WAITING"
    return state                        # no matching arrow: stays


state = "OFF"
print("Events: button, dark, bright · q quits")

while True:
    event = input(state + " > ")
    if event == "q":
        break
    new = next_state(state, event)
    if new != state:
        print("  ", state, "→", new, "| LED", "on" if new == "LIT" else "off")
        state = new
```

Try it: play through the diagram arrow by arrow. Also type events that have **no** arrow in a state (e.g. `dark` in state OFF).

2c · On the board · `main.py`

`next_state` is copied **character for character**. Only where the events come from and what happens on a change is different. The program uses your `hardware.py` from week 4 and is therefore the same for all boards.

```
from hardware import *
import time

THRESHOLD = 30


def next_state(state, event):
    if state == "OFF":
        if event == "button":
            return "WAITING"
    elif state == "WAITING":
        if event == "button":
            return "OFF"
        if event == "dark":
            return "LIT"
    elif state == "LIT":
        if event == "button":
            return "OFF"
        if event == "bright":
            return "WAITING"
    return state


def determine_event():
    if pressed():
        return "button"
    if brightness() < THRESHOLD:
        return "dark"
    return "bright"


state = "OFF"
led(False)

while True:
    new = next_state(state, determine_event())
    if new != state:
        print(state, "→", new)
        state = new
        led(state == "LIT")
    time.sleep_ms(50)
```

Change it · fix the flicker: in week 4 the LED flickered at the border. Use **two** thresholds: "dark" only below 25, "bright" only above 35. In between there is **no** event and the state stays. (Technical term: **hysteresis**, which is how a thermostat works too.)

Solution

```
THRESHOLD_DARK = 25
THRESHOLD_BRIGHT = 35


def determine_event():
    if pressed():
        return "button"
    value = brightness()
    if value < THRESHOLD_DARK:
        return "dark"
    if value > THRESHOLD_BRIGHT:
        return "bright"
    return None                          # grey zone: no event
```

`next_state` stays unchanged, because `None` matches no arrow.

Pro version: transitions as a table

The arrows can also be stored in a **dictionary**. The key is the pair (state, event), the value is the new state. Then the code reads almost exactly like the diagram:

```
TRANSITIONS = {
    ("OFF", "button"): "WAITING",
```

```
    ("WAITING", "button"): "OFF",
    ("WAITING", "dark"): "LIT",
    ("LIT", "button"): "OFF",
    ("LIT", "bright"): "WAITING",
}


def next_state(state, event):
    return TRANSITIONS.get((state, event), state)
```

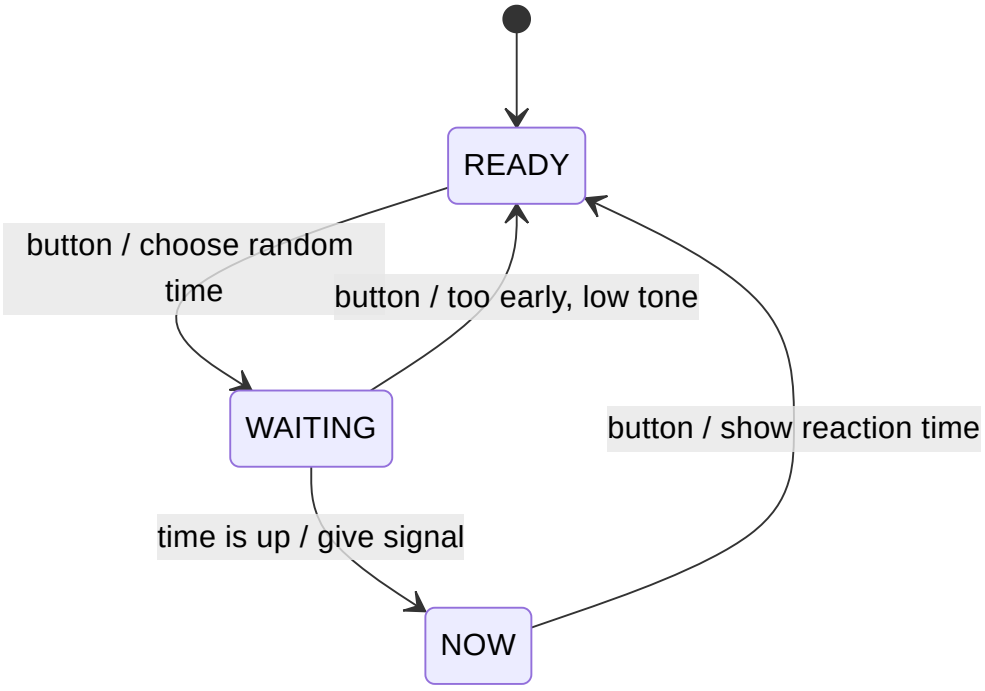
More on dictionaries: Self-study: Data Structures.

German video introducing finite state machines:

▶ *Video: https://www.youtube.com/watch?v=3NroZHXFihE*

3. Reaction game · `reaction.py` (15 min)

A state machine in which **time** is an event. With your `hardware.py`, this program is also the same for all boards. No light sensor in your setup? The `brightness` function is not needed here.




```
from hardware import *
import time
import random

# Test conditions for the experiment
SIGNAL_LED = True
SIGNAL_TONE = True
RANDOM_WAIT = True

state = "READY"
start_time = 0
wait_time = 0
times = []
print("Press the button to start")

while True:
    button = pressed()
    now = time.ticks_ms()
```

```
    if state == "READY":
        if button:
            wait_time = random.randint(1000, 4000) if RANDOM_WAIT else 2000
            start_time = now
            state = "WAITING"

    elif state == "WAITING":
        if button:
            print("Too early!")
            tone(220, 300)
            state = "READY"
        elif time.ticks_diff(now, start_time) >= wait_time:
            start_time = time.ticks_ms()
            if SIGNAL_LED:
                led(True)
            if SIGNAL_TONE:
                tone(880, 100)
            state = "NOW"

    elif state == "NOW":
        if button:
            led(False)
            reaction = time.ticks_diff(now, start_time)
            times.append(reaction)
            print("Attempt " + str(len(times)) + ":", reaction, "ms | average:",
                  sum(times) // len(times), "ms")
            state = "READY"

    time.sleep_ms(5)
```

`time.ticks_ms()` counts milliseconds since the board started, `time.ticks_diff(a, b)` safely computes `a - b`.

Play a few rounds. Compare the program with the diagram: where is each arrow in the code?

4. Experiment: investigating system behaviour (25 min)

An interactive system consists of **input** (button), **processing** (state machine) and **feedback** (LED, tone). How quickly and reliably a person can use it depends a lot on the feedback. You are going to measure that now.

1. **Write down your hypothesis** (before measuring): under which condition will you react fastest? Why?
2. **Measure:** for each condition change the constants at the top of the program, restart, **5 valid attempts** per person. Note the average and the number of "Too early!".

#	SIGNAL_LED	SIGNAL_TONE	RANDOM_WAIT	Average (ms)	"Too early!"
A	True	False	True		
B	False	True	True		
C	True	True	True		
D	True	True	**False**		

3. **Explain:** was your hypothesis right? What could cause the difference between A and B? What happens to the reaction time in D, and what happens to "Too early!"? Why?
4. **Think further:** which feedback would make sense for a person with poor eyesight? For a person with poor hearing?

Background, open only after measuring

Many people react a little faster to a **sound** than to a **light signal**, because the brain processes auditory stimuli faster. With a **fixed waiting time** (D) you learn the rhythm and press **in anticipation**: the

measured time gets shorter, but there are more false starts. The device then no longer measures reaction but guessing. Your values may look different: a few attempts are a small sample, and a blocking `tone()` or the loop pause distorts the result by a few milliseconds. Exactly these influences belong in your explanation.

5. Apply it to your gadget (10 min)

Implement your own state diagram from week 4 using the recipe: first `next_state` (test on the PC with typed events), then `determine_event` with your sensors. Answer as a team:

- Which **feedback** does the person using your gadget get at each transition?
- What happens with an **unexpected input** (button at the wrong moment, sensor covered)?

Put new insights as cards on the Kanban board.

6. Journal task 5 (10 min)

Journal task 5 · Measurement report

```
# P1 · Week 5 · <date>

## Hypothesis
What did you expect before the experiment?

## Measurements
The table A–D with your values.

## Explanation
Was the hypothesis right? How do you explain the differences?
What does this mean for the feedback of your own gadget?

## State machine
An excerpt from `next_state` of your gadget
and the matching part of the state diagram.
```

Commit message: `P1 week 5: state machine and experiment`.

Quiz

Quiz · Week 5

1. According to the state machine recipe, what belongs in the function `next_state`?

a) All outputs such as `led()` and `tone()`.

b) Only the transitions (arrows) of the state diagram.

c) Reading the sensors.

d) The main loop.

2. What does `next_state("LIT", "dark")` return for the night light?

a) `"OFF"`

b) `"WAITING"`

c) `"LIT"`

d) `None`

3. Why does hysteresis (two thresholds) help against flickering?

a) Between the thresholds there is no event, so small fluctuations no longer trigger a change.

b) The sensor measures more accurately.

c) The loop runs more slowly.

d) The LED gets less current.

4. With a fixed waiting time (condition D) the measured reaction time often gets shorter. What is the best explanation?

a) The board computes faster.

b) The tone is louder.

c) People are simply more awake after several attempts.

d) You learn the rhythm and press in anticipation instead of reacting to the signal.

Until next week

- Commit journal task 5.
- `next_state` of your gadget finished and played through on the PC.
- Next week: testing, code review, README and preparing the presentation. The presentation is in **two weeks**.

WEEK 6 · TESTING, CODE REVIEW, PRESENTATION

Today: you make the gadget finished and robust: test the logic automatically, check the device against a test report, review each other's code, write the README and prepare next week's presentation.

What you will have at the end of the lesson

- Your gadget's logic lives in `logic.py` and is checked on the PC by `test_logic.py`.
- `TESTS.md` contains a test report for the real device.
- Another team has reviewed your code, and you have improved the most important points.
- `README.md` in the team repository describes the gadget completely.
- The 3-minute presentation is planned and rehearsed once.

1. Stand-up (5 min)

Additionally today: **what absolutely has to be finished before the review, and what do we drop?** Cards that are no longer realistic go back into the backlog. A small gadget that works reliably beats a big one that crashes during the demo.

2. Testing logic without hardware (20 min)

2a · Three files

Your project gets its final structure:

File	Contents	runs on
`hardware.py`	pins, sensors, actuators (week 4)	board only
`logic.py`	`next_state` and other functions **without** hardware	board **and** PC
`main.py`	main loop: determine event → `next_state` → outputs	board only

`logic.py` for the night light:

```
# logic.py — no imports from hardware!

def next_state(state, event):
    if state == "OFF":
        if event == "button":
            return "WAITING"
    elif state == "WAITING":
        if event == "button":
            return "OFF"
        if event == "dark":
            return "LIT"
    elif state == "LIT":
        if event == "button":
            return "OFF"
        if event == "bright":
            return "WAITING"
    return state
```

In `main.py` you replace the function with `from logic import next_state`. `logic.py` must also be saved on the board.

2b · `test_logic.py`

Every arrow in the state diagram becomes a test case, plus a few cases **without** an arrow. Save `logic.py` and `test_logic.py` in the same folder **on the PC** and run the test in Thonny with the interpreter **Local Python 3**.

```
# test_logic.py — runs on the PC
from logic import next_state

TESTS = [
    # (state,    event,    expected new state)
    ("OFF",     "button", "WAITING"),
    ("WAITING", "button", "OFF"),
    ("WAITING", "dark",   "LIT"),
    ("LIT",     "bright", "WAITING"),
    ("LIT",     "button", "OFF"),
    # events without an arrow: state stays
    ("OFF",     "dark",   "OFF"),
    ("WAITING", "bright", "WAITING"),
    ("LIT",     "dark",   "LIT"),
]

passed = 0
for state, event, expected in TESTS:
    result = next_state(state, event)
    if result == expected:
        passed = passed + 1
        print("ok    ", state, "+", event, "→", result)
    else:
        print("FAILED", state, "+", event, "→", result, "expected:", expected)

print(passed, "of", len(TESTS), "tests passed")
```

Try it:

1. All tests green? Then **deliberately introduce a bug** into `logic.py` (e.g. `return "OFF"` instead of `return "WAITING"`). Does the test find it?
2. Write `logic.py` and `test_logic.py` for **your** gadget. One test case per arrow in your diagram.

<details>
<summary>Short form with <code>assert</code></summary>

Professionals often write tests with `assert`: the line does nothing if the condition holds, and stops with an `AssertionError` if it does not.

```
assert next_state("OFF", "button") == "WAITING"
assert next_state("OFF", "dark") == "OFF"
```
</details>

```
print("all tests passed")
```

2c · Test report on the device

Not everything can be tested on the PC: loose contacts, thresholds, tone volume. That is what the **test report** is for, a table in `TESTS.md`. One person operates the device, one reads out and records. It is based on your acceptance criteria from `REQUIREMENTS.md`.

```
| # | Starting state | Action | Expected | Observed | ok? |
|---|----------------|--------|----------|----------|-----|
| 1 | OFF, room bright | press button | WAITING, LED off | LED off | ✅ |
| 2 | WAITING | hand over sensor | LED on within 1 s | LED on after approx. 0.5 s | ✅ |
| 3 | LIT | press button | OFF, LED off | LED stays on | ❌ |
```

Every ❌ becomes a card on the Kanban board.

3. Code review with another team (20 min)

Two teams swap places and read each other's code (10 min each way). The reviewers write their points as an **issue** in the other team's repository (tab **Issues** → **New issue**) or on paper.

Code review checklist

1. **Does it run?** `main.py` starts without errors, `test_logic.py` is green.
2. **Does the code match the diagram?** Every arrow can be found in `next_state`, and vice versa.
3. **Names:** do variable and function names say what is inside? (`threshold_dark` instead of `x`)
4. **No magic numbers:** pins, thresholds and times are CONSTANTS at the top or in `hardware.py`.
5. **Comments** explain **why**, not what. (`# hysteresis against flicker`, not `# increase x by 1`)
6. **Short functions:** no function is longer than one screen.
7. **One thing** you particularly like.

Points 1–2 are mandatory before the presentation. Improve the rest as far as time allows. More on improving code systematically: Self-study: Code Quality.

4. README: the project's business card (15 min)

The `README.md` is the first thing people see in the repository. Later it is also the basis for the project page in your web portfolio (P5).

```
# <Name of the gadget>

<One sentence: what does it do, for whom?>

![Photo of the gadget](images/gadget.jpg)

## Features
- <the finished user stories, one sentence per story>

## Hardware
| Component | Connection |
|-----------|------------|
| <board> | – |
| <button> | <pin> ↔ GND |

## Getting started
1. Copy `hardware.py`, `logic.py` and `main.py` onto the board.
2. Plug in the board, the program starts automatically.

## State diagram
```

````
```mermaid
stateDiagram-v2
    [*] --> ...
```

## Tests
Run `test_logic.py` on the PC. Test report: [TESTS.md](TESTS.md)

## Team
<names or GitHub usernames, who did what>

## Sources
- <links to guides and code you used>
- Photos: our own
````

Material from others

You may only use code, images or circuit diagrams from the internet if the licence allows it, and you must **name the source**. Photos of people only with their consent. From now on this applies to every project.

5. Prepare the presentation (25 min)

Next week each team has **3 minutes** plus 2 minutes of questions. No slide marathon: the gadget is the star.

Time	Content	Tip
0:30	**Problem and idea:** for whom, what for?	start with an everyday situation
1:00	**Live demo**	practise beforehand who presses what. The demo follows a user story.
0:45	**State diagram** and a **code excerpt** you are proud of	large enough for the back row
0:30	**Learned and difficult:** a real stumbling block and how you solved it	honest, not “everything was great”
0:15	**Outlook:** what would you build next?	

Plan B

Record a **phone video** (30 s) of the working demo today. If a cable gives up on presentation day, show the video and explain what went wrong. That is nothing to be ashamed of, it happens in engineering all the time.

Rehearse once with a stopwatch while another team watches and tells you: what was unclear? Was it under 3 minutes?

6. Journal task 6 (10 min)

Journal task 6 · Bugs and role

```
# P1 · Week 6 · <date>

## A bug we found
How was it noticed (test, test report, code review, chance)?
What was the cause, how did you fix it?

## Code review
The most useful feedback on your code, and one thing
```

```
you learned yourself from the other team's code.

## My role
What is your contribution to the gadget? Link a commit or a file.
```

Commit message: `P1 week 6: tests and review`.

Quiz

Quiz · Week 6

1. Why must `logic.py` not import anything from `hardware.py`?

a) Because Python forbids circular imports.

b) So that the file is smaller.

c) So that `logic.py` can be run and tested on the PC without a board.

d) Because `hardware.py` may only be imported once.

2. The state diagram has 5 arrows. How many test cases does `test_logic.py` need at least?

a) 1

b) 5, one per arrow, and preferably a few more for events without an arrow.

c) As many as `logic.py` has lines.

d) None, if the device works.

3. Which comment is most helpful?

a) `# variable threshold`

b) `# if statement`

c) `# increase threshold by 5`

d) `# two thresholds so the LED does not flicker at the border`

4. What does **not** belong in the test report for the device?

a) How many lines of code the program has.

b) The starting state.

c) What was expected.

d) What was actually observed.

Until next week

- Commit journal task 6.
- Fix the open ❌ from the test report, finish `README.md`.
- Go through the presentation once more at home, put the plan B video into the team repository.

WEEK 7 · REVIEW AND RETROSPECTIVE

Today: the grand finale of P1. You present your gadgets, look at how you worked together in the **retrospective**, and complete your learning journal for this project.

What you will have at the end of the lesson

- Your team has presented its gadget and received feedback.
- You have given written feedback to at least one other team.
- Your team has 1–2 concrete resolutions for P2.
- Your journal for P1 is complete, with project review and self-assessment, and committed.

1. Set up and tech check (10 min)

- Set up the gadget, connect it to power (power bank or USB), **play through it once completely**.
- Have the state diagram and code excerpt ready for projection (team repository open in the browser).
- Keep the plan B video at hand.
- The order of teams is on the board. Each person **draws a team** to give written feedback to.

2. Presentations (45 min)

Each team: **3 minutes of presentation + 2 minutes of questions**, as planned in week 6. The teacher keeps time.

Feedback card (for the team you drew, on paper or as an issue in their team repository):

```
Team: …

I like: …
I wonder: …
What if: …

The demo showed that … (which user story?)
```

What your teacher looks at (product, process, presentation, journal) is in the rubric templates.

3. Retrospective (20 min)

The review looks at the **product**. The **retrospective** looks at the **collaboration**: how did we work, and what will we do differently in the next project? It is not about blame, it is about improvement.

Format **Start · Stop · Continue**, as a team:

Step	Time	What happens
Write silently	5 min	Each person writes sticky notes in three columns: **Start** (what should we begin doing?), **Stop** (what should we stop doing?), **Continue** (what worked well?). One idea per note.

Step	Time	What happens
Present and group	8 min	Read out in turn and stick them up, move similar notes together.
Decide	5 min	Each person has 2 dots to give. The 1–2 notes with the most dots become **resolutions for P2**, phrased as a concrete action: “We do a stand-up every lesson with a photo of the board” instead of “communicate better”.
Record	2 min	Photo of the wall, resolutions into the team repository’s `README.md` under **Retrospective**.

Prompts if you are stuck: did you keep the Kanban board up to date? Were tasks shared fairly? When did you get the most done, when the least, and why? How did merging the code go?

German video on the Start, Stop, Continue format:

▶ *Video: https://www.youtube.com/watch?v=rWz9VsHSpAE*

4. Journal submission (15 min)

Submission means: **the last commit is in**. Nothing is collected (see Learning Journal).

Journal task 7 · Project review P1

File `p1/woche-07.md`:

```
# P1 · Week 7 · Project review · <date>

## Our gadget
One sentence, one photo, link to the team repository.

## My best code
A code excerpt you wrote yourself, and what is good about it.

## Self-assessment
| I can … | 1 | 2 | 3 | 4 | Evidence (link) |
|---------|---|---|---|---|-----------------|
| implement simple algorithms in Python (branching, loop, function, list) | | | | | |
| bring a board with sensor and actuator into operation | | | | | |
| investigate and explain the behaviour of an interactive system by varying inputs and
feedback | | | | | |
| model a device with a state diagram and a flowchart | | | | | |

1 = not yet · 2 = with help · 3 = on my own · 4 = I can explain it to others

## Retrospective
Your team's resolutions for P2, and one personal resolution.

## Feedback
The feedback you got at the presentation in one sentence,
and what you think of it.
```

Checklist before the last commit:

- ☐ `p1/woche-01.md` to `p1/woche-07.md` exist.
- ☐ Journal tasks 1–7 are answered (headings from the templates).
- ☐ Images are in `images/` and are displayed (check the preview).
- ☐ `README.md` links all P1 entries.
- ☐ No passwords, no photos of people without their consent.

Commit message: `P1 week 7: project review, P1 submitted`.

5. Final quiz P1 (5 min)

Final quiz · P1 Code & Gadget

1. What does this program print?

```
values = [4, 8, 6]
total = 0
for v in values:
    if v > 5:
        total = total + v
print(total)
```

a) 18

b) 14

c) 8

d) 3

2. `number = input("Number: ")`. How do you calculate with the input?

a) `number = int(input("Number: "))`

b) `number = input(int("Number: "))`

c) `number = number(input("Number: "))`

d) You don't need to, `input` already returns a number.

3. A roller blind has the states UP, DOWN and MOVING. Which of these is an **event** that triggers a transition?

a) MOTOR

b) MOVING

c) limit switch reached

d) blind

4. A button on a pin with pull-up should toggle the LED **exactly once per press**. What do you need?

a) A longer `time.sleep`.

b) A second pin.

c) `button.value() == 1` instead of `== 0`.

d) Edge detection: only react to the change from not pressed to pressed.

5. How did we split up the code in P1?

a) Everything in `main.py`, so you never have to search.

b) `hardware.py` (board-specific), `logic.py` (testable on the PC), `main.py` (main loop)

c) One file per state.

d) `input.py`, `output.py`, `test.py`

6. You replace the sound of an alarm clock with a short flash of the LED. What are you investigating in terms of human-computer interaction?

a) How different feedback affects the behaviour of the users (e.g. whether they wake up).

b) How fast the processor is.

c) Whether the code gets shorter.

d) How much power the LED needs.

7. What is the purpose of a **retrospective**?

a) To decide the grades for the project.

b) To demonstrate the product.

c) To improve the team's collaboration, with concrete resolutions for the next project.

d) To check the code for bugs.

8. Where on the Kanban board does a card belong whose acceptance criteria are only partly met?

a) Done

b) Backlog, it gets deleted.

c) Anywhere, as long as it is visible.

d) In progress

Outlook: P2 Weather Station

Your board can now measure and react. In P2 it learns to **transmit**: sending measurements over Wi-Fi, storing and analysing them. If you built with an ESP32 or Pico W, you keep using the board, and `hardware.py` simply grows new functions. You take the resolutions from today's retrospective with you.

P2 · WEATHER STATION / IOT

~7 weeks · topic areas 04, 05, 01, 11

Connect sensors, capture → store → analyse data; networking basics, energy & sustainability.

> 🚧 *This page is under construction.*

P3 · AI LAB

~7 weeks · topic areas 03, 01, 11

Decision trees, perceptron, neural networks, generative AI — and the EU AI Act.

> 🚧 *This page is under construction.*

P4 · DATA DETECTIVES

~6 weeks · topic areas 01, 10

SQL & data modelling; encryption, GDPR privacy spheres, OSINT & your data footprint.

> 🚧 *This page is under construction.*

P5 · WEB PORTFOLIO

~7 weeks · topic areas 08, 07, 05

Your agile-built website showcasing all projects; requirements, client-side and server-side parts.

> 🚧 *This page is under construction.*

SELF-STUDY

Learning paths for independent study: curated links and videos, with one **journal task** and one **quiz** per path.

DATA STRUCTURES

> 🚧 *This page is under construction.*

RUNTIME & ALGORITHM ANALYSIS

> 🚧 *This page is under construction.*

CODE QUALITY & REFACTORING

> 🚧 *This page is under construction.*

INTERACTION STYLES (HUMAN-MACHINE)

> 🚧 *This page is under construction.*

LAYER MODEL & NETWORK SERVICES

> 🚧 *This page is under construction.*

PROFESSIONS & ETHICS

> 🚧 *This page is under construction.*

TOOLS

Setup guides for all course tools — for school PCs and your own devices (BYOD). Browser tools work everywhere.

PYTHON & THONNY

Thonny is the Python development environment of this course. It ships with Python, has a built-in shell, a beginner-friendly debugger and can later talk directly to MicroPython boards. One tool for the whole year.

Installation

Windows

1. Download the installer from thonny.org (button “Windows”).
2. Run the installer, keep the default settings. Python is included, nothing else needs to be installed.
3. Start Thonny from the start menu.

On school PCs Thonny is already installed; if not, thonny.org also offers a **portable version** that runs from a folder or USB stick without install rights.

macOS

1. On thonny.org choose the Mac download (Apple Silicon or Intel, depending on your device).
2. Open the `.pkg` file and follow the instructions.
3. Start Thonny from the Applications folder. On first start you may need right-click → Open if macOS checks the origin.

Linux

Easiest via the package manager:

```
sudo apt install thonny      # Debian / Ubuntu / Mint / Pop!_OS
```

Alternatively use the install script from thonny.org.

Browser (fallback)

If nothing can be installed:

- python.microbit.org — Python editor with micro:bit simulator, runs entirely in the browser.
- Wokwi — simulator for Pico, ESP32 and more, see Wokwi & Browser Simulators.

For plain PC Python without a board, any online Python editor that supports `input()` works.

First start

1. On first start Thonny asks for a language: pick yours (changeable later under *Tools* → *Options* → *General*).
2. The top is the **editor** (where you write programs), the bottom is the **shell** (where you try single commands).
3. Type into the shell and press Enter:

```
print("Hello Maker Journey!")
```

4. A program: **File** → **New**, type the code, **File** → **Save** as `test.py`, then the green **play button** (or **F5**).

The most important buttons

Button / key	Meaning
▶ Play (F5)	Run the current file
■ Stop	Abort the running program (important with `while True`)
🐞 Debug (Ctrl+F5)	Run the program step by step, watch variables
View → *Variables*	Shows all variables and their values while the program runs
View → *Assistant*	Explains error messages in plain language

If a program stops responding

Press the stop button or click into the shell and press **Ctrl+C**. With an endless loop this is normal, not an error.

Videos

Short introduction to the interface (German):

▶ *Video: https://www.youtube.com/watch?v=93J6Sxk55mg*

First steps with Python in Thonny (German):

▶ *Video: https://www.youtube.com/watch?v=k7CLrRkzC5o*

Later in the course

Thonny is the tool for **P1 and P2**. From **P3** the course switches to **Visual Studio Code**, which is the better home for notebooks, data analysis, SQL and later web development with Git. A separate setup guide follows at the start of P3.

From P1 week 3 you connect Thonny to your board: *Tools* → *Options* → *Interpreter* → select MicroPython. Guide under MicroPython Boards.

MICROPYTHON BOARDS (MICRO:BIT, PICO, ESP32, ARDUINO)

MicroPython is Python for small computers without screen or keyboard, so-called **microcontrollers**. You write the same Python code as on the PC (`if`, `while`, functions, lists), plus modules such as `machine` or `microbit` that control pins, LEDs and sensors. You program in Thonny, the board is connected to the PC via USB.

You need this page from **P1 week 3**. No board at hand? The same code runs in the Wokwi simulator.

Which board do I have?

Board	How to recognise it	MicroPython?
micro:bit	Pocket-sized, 5×5 LED display, buttons A and B. **V2** has a golden touch logo and a speaker on the back.	yes
Raspberry Pi Pico	Green strip, printed “Raspberry Pi Pico” (**Pico W** / **Pico 2 W** with a silver Wi-Fi chip cover), button **BOOTSEL**.	yes
ESP32	Usually a black board with a silver metal cover “ESP32-WROOM”, two buttons **EN** and **BOOT**. Many manufacturers, pins labelled differently.	yes
Arduino Nano ESP32	Small Arduino board, printed “Nano ESP32”.	yes
Arduino UNO R3 / UNO R4	Large Arduino board with header sockets.	**no**, see tab “Arduino”

The strengths of each board are listed in week 2.

What you need

- A **USB data cable**. Many cables from phone chargers can only charge, not transfer data. If the board is not detected, this is the most common cause. Mark working cables with tape.
- For Pico, ESP32 and Nano ESP32: a **breadboard**, **jumper wires**, LEDs, **resistors** 220 Ω and 10 kΩ, a **push button**.
- For micro:bit: nothing, everything you need is built in. For your own components: **crocodile clips** or an **edge connector breakout**.

Part 1 · Install MicroPython and connect Thonny

You do this **once per board**. Afterwards: plug in the board, start Thonny.

micro:bit

With Thonny:

1. Connect the micro:bit via USB. A drive **MICROBIT** appears on the PC.
2. Thonny: **Tools → Options → Interpreter → MicroPython (BBC micro:bit)**.
3. Click **Install or update MicroPython** at the bottom right, choose the MICROBIT drive, **Install**. The yellow LED on the back flashes, then the board restarts.
4. **OK**. The shell shows `>>>`. Test:

```
from microbit import *
display.scroll("Hello")
```

Without installing anything (browser): open python.microbit.org, write code, transfer it with **Send to micro:bit** (Chrome or Edge, connects directly via USB). In other browsers click **Save** and drag the `.hex` file onto the MICROBIT drive. The editor has a built-in simulator, see Wokwi & Browser Simulators.

Raspberry Pi Pico

1. Hold the **BOOTSEL** button, **then** plug in the USB cable, then release. A drive **RPI-RP2** (Pico 2: **RP2350**) appears on the PC.
2. Thonny: **Tools → Options → Interpreter → MicroPython (Raspberry Pi Pico)**.
3. Click **Install or update MicroPython** at the bottom right.
4. **Target volume:** the RPI-RP2 drive. **Variant:** pick exactly your model: *Pico / Pico H*, *Pico W / Pico WH*, *Pico 2* or *Pico 2 W*. The wrong variant runs without Wi-Fi or not at all.
5. **Install**. After copying, the drive disappears and the board restarts.
6. Close the window, **OK**. The shell shows `>>>`. Test:

```
from machine import Pin
Pin("LED", Pin.OUT).value(1)    # onboard LED lights up
```

No drivers are needed on Windows 10/11, macOS and Linux.

ESP32

1. Connect the ESP32 via USB.
2. **Check the driver (Windows only, first time only):** In Device Manager under *Ports (COM & LPT)* a new **COM port** must appear. If it is missing, look at the small chip next to the USB socket and install the driver:
   - printed **CP210x** (Silicon Labs) → CP210x driver
   - printed **CH340** / **CH9102** (WCH) → CH340 driver
3. Thonny: **Tools → Options → Interpreter → MicroPython (ESP32)**, under *Port* choose the new port (`COM3`, `/dev/ttyUSB0`, `/dev/cu.usbserial-…`).
4. Click **Install or update MicroPython (esptool)** at the bottom right.
5. **Target port:** your port. **MicroPython family:** *ESP32*. **Variant:** *Espressif • ESP32 / WROOM*. If you have an ESP32-**S3** or **C3**, choose the matching family.
6. **Install**. If it hangs at `Connecting...`: hold the **BOOT** button until percentages appear.
7. Close the window, **OK**. The shell shows `>>>`. Test:

```
from machine import Pin
Pin(2, Pin.OUT).value(1)        # onboard LED (on most DevKits)
```

Arduino

Arduino UNO R3 and UNO R4: There is **no official MicroPython** for these boards. The course is in Python, so there are two ways:

1. **Simulator:** create an **ESP32** project in Wokwi and use the course's ESP32 code. This is equivalent: you build the same circuit virtually.
2. **Swap the board:** get an ESP32, Pico or **Arduino Nano ESP32** from your teacher.

Arduino Nano ESP32:

1. Download and start the Arduino MicroPython Installer.
2. Connect the Nano ESP32. If the installer does not detect the board: press the **reset** button twice quickly.
3. Select the board, **Install MicroPython**.
4. Thonny: **Tools → Options → Interpreter → MicroPython (ESP32)**, choose the port, **OK**. Test:

```
from machine import Pin
Pin(48, Pin.OUT).value(1)       # onboard LED (D13)
```

Code as in the ESP32 tab, but with **different pin numbers**, see part 3.

Part 2 · Working with the board in Thonny

As soon as Thonny is connected to the board:

- The **shell** talks to the **board**. `print(1 + 1)` is computed by the microcontroller.
- **▶ Play (F5)** sends the file in the editor to the board and runs it there. It is **not** saved: after unplugging it is gone.
- **■ Stop** (or **Ctrl+C** in the shell) aborts the program. A second click on Stop restarts the board (soft reset).

Files on the board

View → Files shows the files on your PC at the top and the files on the **MicroPython device** at the bottom.

What	How
Save a file to the board	**File → Save as → MicroPython device**, type the name
Upload a file from the PC	In the upper (PC) part of the files pane right-click the file → **Upload to /**
Get a file from the board	Right-click the file on the device → **Download to …**
Delete a file	Right-click → **Delete**

Two file names have a special meaning:

File	Meaning
`main.py`	Starts automatically as soon as the board gets power, also from a power bank. This is your gadget program.
`boot.py`	Starts even before `main.py`. Do not change it in this course.

Your own modules such as `hardware.py` (from week 4) must be **on the board** so that `import hardware` works.

Code belongs in the repository, not only on the board

The board is not a storage place. Save every file on the PC too and upload it to the team repository. A

board can get lost, swapped or re-flashed.

Part 3 · Pinout and course wiring

All programs in the course use the same wiring per board. Stick to it and the examples run without changes.

⚠ Power off while wiring

- Only plug in or move components when the USB cable is **unplugged**.
- All boards work with **3.3 V**. Never connect **5 V** to an input pin.
- Always use a **series resistor** (220 Ω) with LEDs. The **long leg** of the LED goes towards the pin, the short one to GND.
- Never connect **3V3** and **GND** directly (short circuit).

micro:bit

For P1 there is **nothing to wire**: buttons A/B, 5×5 display, light sensor (the display itself), accelerometer, temperature, microphone and speaker (V2) are built in.

Course function	micro:bit
Button	`button_a`, `button_b`
LED	pixel on `display`
Buzzer	built-in speaker, `music.pitch(...)`
Light sensor	`display.read_light_level()` (0–255)
Tilt	`accelerometer.get_x()`

Your own components connect with crocodile clips to the large rings **0**, **1**, **2**, **3V** and **GND**:

```
from microbit import *

pin0.write_digital(1)        # external LED on pin 0 (with 220 Ω) on
value = pin1.read_analog()   # analog value on pin 1, 0–1023
print(value)
```

The small pins along the edge are only reachable with a breakout board. Many of them belong to the LED display. Stick to 0, 1 and 2.

Raspberry Pi Pico

Careful: the number in the code is the **GP number**, not the number of the physical pin. `Pin(15)` is **GP15**, which is physical pin 20.

Course function	GP pin	Physical pin	Connection
Button	**GP14**	19	button between GP14 and GND (pin 18)
LED	**GP15**	20	GP15 → 220 Ω → LED long leg, short leg → GND
Onboard LED	`"LED"`	–	built in, no wiring

Course function	GP pin	Physical pin	Connection
Buzzer (passive)	**GP16**	21	GP16 → buzzer → GND (pin 23)
Light sensor / pot	**GP26** (ADC0)	31	3V3 → LDR → GP26 → 10 kΩ → GND
3.3 V	3V3(OUT)	36	plus for sensors
GND	GND	3, 8, 13, 18, 23, 28, 33, 38	all GNDs are the same

Analog inputs exist only on **GP26**, **GP27** and **GP28**. Official pinout to print: Pico, Pico W.

The breadboard: the Pico sits across the centre groove, each row of five next to a pin is connected to it. The long rails along the edge (red/blue) are handy for 3V3 and GND.

ESP32

Applies to the common **ESP32 DevKit** (30 or 38 pins). The number in the code is the **GPIO number**, usually printed on the board as `D18`, `G18`, `IO18` or just `18`.

Course function	GPIO	Connection
Button	**4**	button between GPIO 4 and GND
LED	**18**	GPIO 18 → 220 Ω → LED long leg, short leg → GND
Onboard LED	**2**	built in on most DevKits (not all)
Buzzer (passive)	**19**	GPIO 19 → buzzer → GND
Light sensor / pot	**34**	3V3 → LDR → GPIO 34 → 10 kΩ → GND

Pins with a catch:

Pins	Note
6–11	**Never use**, they belong to the internal flash memory.
0, 2, 12, 15	Affect booting. Only use if needed (GPIO 2 for the onboard LED is fine).
34, 35, 36 (VP), 39 (VN)	**Input only**, no `Pin.PULL_UP`. Good for sensors, bad for buttons.
32–39	Analog inputs that also work with **Wi-Fi** on (important for P2). Other ADC pins (e.g. 25, 26, 27) do not work while Wi-Fi is active.

Always configure analog inputs with `adc.atten(ADC.ATTN_11DB)`, otherwise the range ends at about 1 V.

Arduino

Arduino Nano ESP32: the board shows Arduino names (`D2`, `A0`), but MicroPython uses the **GPIO numbers** of the ESP32-S3. Translation:

Arduino	D0	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10	D11	D12	D13
GPIO	44	43	5	6	7	8	9	10	17	18	21	38	47	48

Arduino	A0	A1	A2	A3	A4	A5	A6	A7
GPIO	1	2	3	4	11	12	13	14

Course wiring (chosen so that the ESP32 code changes as little as possible):

Course function	Connection on the Nano	In code	Difference to ESP32 code
Button	**A3** ↔ GND	`Pin(4, …)`	none
LED	**D9** → 220 Ω → LED → GND	`Pin(18, …)`	none
Onboard LED	D13	`Pin(48, …)`	instead of 2
Buzzer	**D8** → buzzer → GND	`PWM(Pin(17))`	instead of 19
Light sensor / pot	3V3 → LDR → **A0** → 10 kΩ → GND	`ADC(Pin(1))`	instead of 34

So in `hardware.py` (week 4) you only change the numbers 19 → 17 and 34 → 1. The board works with **3.3 V**, not 5 V like the UNO.

UNO R3 / UNO R4: no MicroPython, see part 1.

Placing the button correctly

A four-legged push button always connects two pairs. It works reliably if you use **two diagonally opposite** legs. The button sits across the centre groove of the breadboard.

Part 4 · More components

Your gadget may need more than LED, button and light sensor. Each component becomes **one small function** in `hardware.py`, the rest of the code stays the same (see week 4).

Suggested pins that do not clash with the course wiring:

Component	Pico	ESP32	Nano ESP32	Power
Motion sensor (PIR, HC-SR501)	GP17	27	D4 → 7	5 V
Ultrasonic distance: Trig	GP18	25	D5 → 8	5 V (HC-SR04) / 3.3 V (HC-SR04**P**)
Ultrasonic distance: Echo	GP19	26	D6 → 9	see warning
Servo (SG90)	GP20	13	D7 → 10	5 V
Temperature/humidity (DHT11/DHT22)	GP21	33	D10 → 21	3.3 V
OLED display (I²C): SDA / SCL	GP4 / GP5	21 / 22	A4 → 11 / A5 → 12	3.3 V
second analog value	GP27	35	A1 → 2	3.3 V

5 V is available on pin **VBUS** (Pico, pin 40), **5V/VIN** (ESP32) or **VBUS** (Nano ESP32) while the board is powered via USB.

The examples below use the **Pico** pins. For other boards insert the number from the table. On the micro:bit most of these components are awkward (few pins, no 5 V). Use the built-in sensors there instead.

Motion sensor (PIR)

The PIR returns 1 as long as it detects motion. The two knobs set sensitivity and hold time. After power-on it needs about a minute to calibrate.

```
from machine import Pin

_pir = Pin(17, Pin.IN)


def motion():
    """True as long as motion is detected."""
    return _pir.value() == 1
```

Ultrasonic distance sensor (HC-SR04)

```
from machine import Pin, time_pulse_us
import time

_trig = Pin(18, Pin.OUT)
_echo = Pin(19, Pin.IN)


def distance_cm():
    """Distance in cm, or -1 if nothing was measured."""
    _trig.value(0)
    time.sleep_us(2)
    _trig.value(1)
    time.sleep_us(10)
    _trig.value(0)
    duration = time_pulse_us(_echo, 1, 30000)   # wait at most 30 ms
    if duration < 0:
        return -1
    return duration * 0.0343 / 2                # sound: 343 m/s, there and back
```

Servo (SG90)

Brown = GND, red = **5 V**, orange = signal. A servo draws a lot of current: if the board restarts as soon as the servo moves, USB power is not enough. Then power the servo from a separate source (connect the GNDs).

```
from machine import Pin, PWM

_servo = PWM(Pin(20))
_servo.freq(50)


def servo(angle):
    """Turns the servo to 0–180 degrees."""
    # 0.5 ms (0°) to 2.5 ms (180°) out of 20 ms
    _servo.duty_u16(1638 + angle * (8192 - 1638) // 180)
```

Temperature and humidity (DHT11 / DHT22)

The `dht` module is already included on Pico and ESP32. The sensor may be read at most **once per second** (DHT22: every 2 s).

```
from machine import Pin
import dht

_dht = dht.DHT11(Pin(21))      # for DHT22: dht.DHT22(...)


def temperature():
    _dht.measure()
```

```
    return _dht.temperature()   # °C


def humidity():
    _dht.measure()
    return _dht.humidity()      # %
```

OLED display (SSD1306, 128×64, I²C)

Install the driver on the board once: Thonny **Tools → Manage packages**, search for `ssd1306`, **Install**.

```
from machine import Pin, I2C
import ssd1306

_i2c = I2C(0, sda=Pin(4), scl=Pin(5))     # ESP32: I2C(0, sda=Pin(21), scl=Pin(22))
_oled = ssd1306.SSD1306_I2C(128, 64, _i2c)


def show_text(line1, line2=""):
    _oled.fill(0)
    _oled.text(line1, 0, 0)
    _oled.text(line2, 0, 16)
    _oled.show()
```

If `I2C(...)` reports an error or the display stays dark: `print(_i2c.scan())` must print `[60]`. An empty list means SDA/SCL are swapped or not connected.

Troubleshooting

Problem	Likely cause	Fix
Thonny finds no device, no port in the list	charge-only cable, missing driver (ESP32)	other cable, other USB port, driver from part 1
`Device is busy or does not respond`	`main.py` runs an endless loop	press **Stop** several times. If that does not help: unplug, plug in, press Stop immediately
`ImportError: no module named 'hardware'`	`hardware.py` is only on the PC	save the file **to the board** (part 2)
`ValueError: invalid pin`	physical pin number instead of GP/GPIO number	check the pinout (part 3)
LED stays dark	LED reversed, wrong pin, GND missing	long leg to the pin, compare wiring with the table
Button always returns 0 or always 1	`Pin.PULL_UP` missing, button rotated, pin 34–39 on ESP32	set pull-up, diagonal legs, different pin
Analog value always 0 or always maximum	wrong pin, not an ADC pin, `atten` missing (ESP32)	Pico: GP26–28. ESP32: 32–39 with `ATTN_11DB`
Board restarts when buzzer/servo/motor starts	USB supplies too little current	separate power supply for the load
Program does not run without PC	file is not called `main.py` or only on the PC	save as `main.py` to the board
micro:bit: drive is called **MAINTENANCE**	reset was pressed while plugging in	unplug, plug in again without pressing a button

If nothing works any more

Simply **re-install** MicroPython (part 1). This deletes all files on the board. That is why code belongs in the repository.

Further reading

- MicroPython documentation: *quick reference* for RP2/Pico and ESP32
- micro:bit MicroPython documentation
- Arduino Nano ESP32 and MicroPython

WOKWI & BROWSER SIMULATORS

A **simulator** shows a virtual board with LEDs, buttons and sensors in the browser. Your MicroPython code runs in it **just like on the real board**. You don't need to install anything, only a browser.

Simulator	Boards	Used for
Wokwi	Raspberry Pi Pico, ESP32	your own circuits with breadboard components, `hardware.py`, later Wi-Fi (P2)
python.microbit.org	micro:bit	editor and simulator in one, sensors via sliders

When to simulate?

The simulator is **not a stopgap** but an equivalent tool:

- **No board available:** at home, when ill, when the team board is with someone else.
- **Arduino UNO R3 / R4:** these boards cannot run MicroPython. Your team builds the circuit in Wokwi with an **ESP32** (see MicroPython Boards).
- **Before wiring:** build and test the circuit virtually first. What does not work in the simulator will not work on the breadboard either.
- **For sharing:** a Wokwi link shows code **and** circuit. Ideal for the team repository and for questions to your teacher.

Part 1 · Wokwi: start a project

1. Open an empty MicroPython project:
   - New Pico project (MicroPython)
   - New ESP32 project (MicroPython)
2. On the **left** is the code (`main.py`) and the circuit description (`diagram.json`), on the **right** the simulation.
3. Press the green ▶ **button** above the simulation. The program starts, `print` output appears in the console at the bottom right.
4. ■ ends the simulation. If you change the code, ▶ restarts it.

Try it without wiring anything (Pico, onboard LED):

```
from machine import Pin
import time

led = Pin("LED", Pin.OUT)       # if "LED" does not work: Pin(25, Pin.OUT)

for i in range(10):
    led.value(not led.value())
    time.sleep(0.5)
print("Done")
```

The ESP32 board in Wokwi (DevKitC) has **no onboard LED**. Connect an external LED to GPIO 18 right away, see part 3.

REPL in Wokwi

When `main.py` has finished, the console shows `>>>`. There you can try single commands like in the

Thonny shell. If an endless loop is running, click into the console and press **Ctrl+C**.

Part 2 · Adding and wiring components

What	How
Add a component	blue **+** button above the simulation, search for the component (e.g. *LED*, *Resistor*, *Pushbutton*, *Buzzer*, *Potentiometer*, *Photoresistor*)
Move a component	drag it
Rotate a component	click it, press **R**
Properties (colour, resistance)	click the component, change them in the pop-up
Draw a wire	click a pin, then the target pin. Clicks in between set bend points.
Wire colour	click the wire, choose a colour. Convention: **black = GND**, **red = 3V3**
Delete	click it, press **Delete**

Hover over a pin and Wokwi shows its name (`GP15`, `GND.3`, `A`, `C`). On the LED, **A** (anode) is the long leg (+), **C** (cathode) the short one.

Wokwi only draws what is in `diagram.json`. So the mouse editor and the text in `diagram.json` change the same thing.

Part 3 · The course wiring in Wokwi

Rebuild the circuit from weeks 3 and 4. Then the course's `hardware.py` runs **without changes**.

Raspberry Pi Pico

Component	Connection
Button	GP14 ↔ GND
LED + resistor 220 Ω	GP15 → resistor → LED **A**, LED **C** → GND
Buzzer	GP16 → buzzer **2** (+), buzzer **1** (−) → GND
Potentiometer (instead of light sensor)	**VCC** → 3V3, **SIG** → GP26, **GND** → GND

Shortcut via `diagram.json`: open the `diagram.json` tab. Insert the following lines **right after** `"parts": [` (before the board's entry):

```
    { "type": "wokwi-pushbutton", "id": "btn1", "top": 0, "left": 150, "attrs": {
"color": "green" } },
    { "type": "wokwi-resistor", "id": "r1", "top": 80, "left": 150, "attrs": { "value":
"220" } },
    { "type": "wokwi-led", "id": "led1", "top": 50, "left": 250, "attrs": { "color":
"red" } },
    { "type": "wokwi-buzzer", "id": "bz1", "top": 140, "left": 150, "attrs": {} },
    { "type": "wokwi-potentiometer", "id": "pot1", "top": 240, "left": 150, "attrs": {}
},
```

And these lines **right after** `"connections": [`:

```
    [ "pico:GP14", "btn1:1.l", "green", [] ],
    [ "btn1:2.l", "pico:GND.4", "black", [] ],
    [ "pico:GP15", "r1:1", "orange", [] ],
    [ "r1:2", "led1:A", "orange", [] ],
    [ "led1:C", "pico:GND.5", "black", [] ],
    [ "pico:GP16", "bz1:2", "purple", [] ],
    [ "bz1:1", "pico:GND.6", "black", [] ],
    [ "pot1:VCC", "pico:3V3", "red", [] ],
    [ "pot1:SIG", "pico:GP26", "blue", [] ],
    [ "pot1:GND", "pico:GND.8", "black", [] ],
```

Then go back to the simulation and drag the components into a tidy layout.

ESP32

Component	Connection
Button	GPIO 4 ↔ GND
LED + resistor 220 Ω	GPIO 18 → resistor → LED **A**, LED **C** → GND
Buzzer	GPIO 19 → buzzer **2** (+), buzzer **1** (–) → GND
Potentiometer (instead of light sensor)	**VCC** → 3V3, **SIG** → GPIO 34, **GND** → GND

Shortcut via `diagram.json`: open the `diagram.json` tab. Insert the following lines **right after** `"parts": [` (before the board's entry):

```
    { "type": "wokwi-pushbutton", "id": "btn1", "top": 0, "left": 200, "attrs": {
"color": "green" } },
    { "type": "wokwi-resistor", "id": "r1", "top": 80, "left": 200, "attrs": { "value":
"220" } },
    { "type": "wokwi-led", "id": "led1", "top": 50, "left": 300, "attrs": { "color":
"red" } },
    { "type": "wokwi-buzzer", "id": "bz1", "top": 140, "left": 200, "attrs": {} },
    { "type": "wokwi-potentiometer", "id": "pot1", "top": 240, "left": 200, "attrs": {}
},
```

And these lines **right after** `"connections": [`:

```
    [ "esp:4", "btn1:1.l", "green", [] ],
    [ "btn1:2.l", "esp:GND.2", "black", [] ],
    [ "esp:18", "r1:1", "orange", [] ],
    [ "r1:2", "led1:A", "orange", [] ],
    [ "led1:C", "esp:GND.2", "black", [] ],
    [ "esp:19", "bz1:2", "purple", [] ],
    [ "bz1:1", "esp:GND.3", "black", [] ],
    [ "pot1:VCC", "esp:3V3", "red", [] ],
    [ "pot1:SIG", "esp:34", "blue", [] ],
    [ "pot1:GND", "esp:GND.1", "black", [] ],
```

Then go back to the simulation and drag the components into a tidy layout.

Arduino teams (UNO R3 / R4) use exactly this ESP32 circuit.

⚠ Errors in diagram.json

Every entry ends with a comma, **except the last one** in the list. If the list was empty before (`[]`), delete the comma after the last inserted line. Wokwi underlines errors in red. If the board in your file is not called `pico` or `esp` (see `"id"` in the board's entry), replace the name in the connections.

Light sensor instead of potentiometer

Wokwi has a light sensor module (*Photoresistor sensor*, pins **VCC** → 3V3, **GND** → GND, **AO** → GP26 or GPIO 34). Careful: with this module the value **drops** when it gets brighter, exactly the opposite of the course circuit. Then calculate `100 - …` in `hardware.py`:

```
def brightness():
    return 100 - _sensor.read_u16() * 100 // 65535
```

For the course the **potentiometer** is simpler: turn it = brighter/darker.

Part 4 · Several files: `hardware.py` and `main.py`

From week 4 your program consists of several files. Wokwi copies **all files** of the project onto the virtual board, `main.py` starts automatically.

1. Next to the file tabs click the small **arrow ▾ → New file...**
2. Type the name, e.g. `hardware.py`, paste the code.
3. In `main.py`, `import hardware` as usual.

The same menu lets you **upload** files (*Upload file(s)...*) and **download** the whole project as a ZIP.

Part 5 · During the simulation

Component	How to operate it
Button	click and hold. **Ctrl**/**Cmd**+click keeps it pressed.
Potentiometer	drag the knob, or click it and press ← →
Light sensor	click it, move the *lux* slider
Buzzer	sound comes from your computer's speaker (check the volume)

You see readings with `print()` in the console. Wokwi has no plotter like Thonny. For the threshold in week 4 the numbers in the console are enough.

Part 6 · Save, share, hand in

1. **Save** (or **Ctrl+S**). The first time, Wokwi asks you to sign in. Easiest: **sign in with GitHub**, you already have that account.
2. After saving, the project has its own address, e.g. `https://wokwi.com/projects/123456789`. This address is your **link for sharing**.
3. For the team repository:
   - put the **Wokwi link** into `README.md`,
   - still **upload** the files (`main.py`, `hardware.py`, ...), because the repository is the source, not Wokwi,
   - take a **screenshot** of the circuit for the journal.

Who can see my project?

Anyone who knows the link can see a saved Wokwi project. Do not put personal data in the code or project name. A good project name: `p1-gadget-<teamname>`.

If two of you work on the same project: Wokwi cannot be edited simultaneously. One person saves, the other opens the link and clicks **Save a copy**. Code is merged in the team repository.

micro:bit: python.microbit.org

The micro:bit Python Editor has the simulator built in.

1. Open python.microbit.org. Write code on the left.
2. On the right, press the ▶ **button** above the virtual micro:bit.
3. Click buttons **A** and **B** and the logo in the picture.
4. Below the simulator you set the **sensors**: light level, temperature, sound level, acceleration and gestures (*shake*, *tilt*, ...).
5. `print` output appears below the simulator (*Serial*).

Code for the simulator is **the same** as for the real micro:bit. **Send to micro:bit** (Chrome/Edge) puts it directly onto the board.

Saving: the editor remembers your project in the browser, but only on this device. **Save** downloads a `.hex` file (for sharing or flashing). The menu next to **Save** also has *Save Python script* for the `.py` file. That one belongs in the team repository.

Several files: use **Open...** to open another `.py` file such as `hardware.py` and **add it to the project as a new file**. It then appears in the *Project* area.

Limits of the simulator

A simulator is a **model** of reality. Some things it does not reproduce:

In the simulator	On the real board
sensors deliver exact, steady values	values fluctuate, check thresholds again on the board
a reversed LED simply does not light	same, but a short circuit can **destroy** components
power is unlimited	servos and motors can make the board restart
timing roughly realistic	reaction games and tone lengths may feel slightly different
Wi-Fi via the network `Wokwi-GUEST` (no password, ESP32 and Pico W only)	real school Wi-Fi, see P2

So the rule is: **develop in the simulator, final check on the board.** Arduino teams demo their gadget in Wokwi at the review.

Other simulators

- **Wokwi** with Arduino UNO: here you program in **C++** instead of Python. Not part of P1, just for the curious.
- **MakeCode**: micro:bit with blocks. You may know it from lower secondary. In this course we write Python.

GIT & GITHUB

GitHub is where your learning journal and later your teams' code live. **Git** is the tool behind it that records every change as a **commit**. In week 1 you work in the browser only. Working with Git locally comes as soon as you share code in a team.

Part 1 · Create an account

1. Open github.com/signup.
2. Choose an email address (school address if you have one), a password and a **username**. The username is publicly visible: no full real name required, but such that your teacher can identify you, e.g. `firstname-lastname-initials` or as agreed in class.
3. Enter the confirmation code from the email.

⚠ Password

Write the password down in a safe place. A forgotten GitHub password costs half an hour. If the school has a password manager, use it.

Part 2 · Create the learning journal repository

With invitation link (Classroom 50)

The class uses **Classroom 50**, a free tool that creates a private repository for each person in the school's organisation.

1. **Report your username:** tell your teacher your GitHub username (list on the board or a form). Only then can they enable you.
2. Open classroom50.org, click **Sign in with GitHub** and allow Classroom 50 in the window (**Authorize**).
3. Open the teacher's **classroom link** (accept the organisation invitation if asked).
4. Open the **assignment link** `lernjournal`. The page shows your username and the name of the repository you will get.
5. Click **Accept assignment**. A checklist shows the progress, it takes up to a minute.
6. Click **Open repository** and **bookmark** the page.

Your repository is named after the pattern `<class>-lernjournal-<username>`. **Do not rename it**, otherwise Classroom 50 no longer finds it. It is private: only you and your teacher see it. The folder structure and a template are already inside.

Create it yourself

1. Top right **+** → **New repository**.
2. Repository name: `lernjournal`. Visibility: **Private**.
3. Tick **Add a README file**. Then **Create repository**.
4. Add your teacher as a reader: **Settings** → **Collaborators** → **Add people**, enter the teacher's GitHub username.
5. Create the folder structure: **Add file** → **Create new file**, type `p1/woche-01.md` as the file name (the slash creates the folder), paste the content of the template, **Commit changes**.

Part 3 · Edit a file in the browser and commit

1. In the repository click the file, e.g. `README.md`.
2. Top right the **pencil icon** (Edit this file).
3. Change the text. Use the **Preview** tab to check how the Markdown looks.
4. Top right **Commit changes...**
5. Enter a short **commit message** that says what you changed, e.g. `README: added name and class`. Then **Commit changes**.

That is the whole cycle: **edit → preview → commit**. Every commit is a point on the timeline. Under **Commits** (or the clock icon) you see all previous versions and can look at any old state.

Create a new file

Add file → Create new file. Put a folder with a slash into the file name: `p1/woche-02.md`.

Upload an image

1. **Add file → Upload files**.
2. Drag the screenshot in. Name it sensibly first, e.g. `p1-w01-switch.png`.
3. At the bottom **Commit changes**. The image is now in the repository (put it into the `images/` folder by navigating into that folder first).
4. Embed it in the journal entry: `![My switch](../images/p1-w01-switch.png)`

> **Taking a screenshot**
>
> Windows: **Win+Shift+S** · macOS: **Cmd+Shift+4** · Linux (GNOME): **Print** key. The image lands in the clipboard or in the Pictures folder.

The three terms you need today

Term	Meaning
Repository (repo)	A project folder with the complete history of all changes.
Commit	A saved state with message, time and author. Immutable.
Commit message	One sentence saying what changed. For you in three months and for your teacher.

Later: Git locally

As soon as teams write code together (from P1 sprint 1), you work with Git on your own computer: clone, change, commit, push, and work in parallel with **branches**. This guide will be extended then. Until then the browser is enough.

MARKDOWN CHEAT SHEET

Markdown is text with a few special characters for formatting. Your learning journal is written in Markdown, so is this book, and README files on GitHub anyway. You only need this handful of rules.

The essentials

You write	You get
`# Heading`	large heading (one hash = level 1, `##` = level 2, `###` = level 3)
`**bold**`	**bold**
`*italic*`	*italic*
`- item`	bullet list (start every line with `-`)
`1. step`	numbered list
`` `code` ``	`code` inside text
`[text](https://…)`	link
`![description](../images/picture.png)`	embedded image
`---`	horizontal rule

Separate paragraphs with a **blank line**. A single line break is not enough.

Code blocks

Multi-line code goes between three backticks, followed by the language for colours:

````
```python
state = "OFF"
print(state)
```
````

becomes

```python
state = "OFF"
print(state)
```

A complete example

```
# P1 · Week 1 · 2026-09-14

## Done
- Started Thonny, wrote `hello.py` and `switch.py`
- Created GitHub account and learning journal

## Worked
- The switch toggles between **OFF** and **ON**
```

```
## Stuck on
- `while True` ran forever until I found the stop button

## Evidence
![My switch](../images/p1-w01-switch.png)
```

Preview

The GitHub editor has a **Preview** tab. There you see immediately how your text looks. In VS Code it is **Ctrl+Shift+V** later on.

More

- Diagrams right inside Markdown: Diagrams with Mermaid
- Basic writing and formatting syntax on GitHub Docs
- CommonMark reference with an interactive 10-minute tutorial

---

DIAGRAMS WITH MERMAID

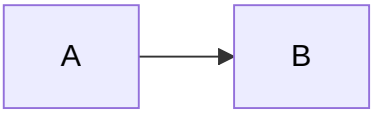
Mermaid draws diagrams from text. You describe in a few lines which boxes there are and which arrows connect them, and Mermaid turns that into a picture. Because the diagram is just text, it lives right inside your Markdown file, can be versioned with Git and is changed in seconds. No drawing program, no screenshot that goes out of date.

How to write a Mermaid block

A Mermaid diagram is a code block with the language `mermaid`:

````
```mermaid
flowchart LR
    A --> B
```
````

becomes



The **first line** in the block says which kind of diagram you want. For this book you need two: `flowchart` and `stateDiagram-v2`.

Where is Mermaid displayed?

GitHub and **this book** show the block as a picture. **Thonny** and many editors only show the text. To try things out, use the online editor mermaid.live: type on the left, see the picture on the right straight away.

Flowchart: `flowchart`

Direction

Right after `flowchart` comes the direction:

Code	Direction
`LR`	left to right
`TD`	top to bottom

Nodes

Every node has an **ID** (short, no spaces or special characters) and a **label**. The brackets around the label decide the shape:

You write	Shape	Used for
`A[Text]`	rectangle	step, component
`A([Text])`	rounded	start, end
`A{Text}`	diamond	decision, question
`A((Text))`	circle	the device itself

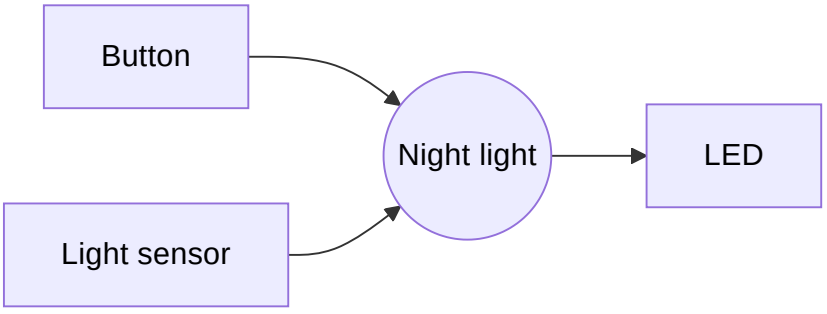
Once a node has a label, the ID alone is enough after that.

Arrows

You write	Meaning
`A --> B`	arrow
`A -->|yes| B`	arrow with a label
`A --- B`	line without a tip

Example 1: inputs → gadget → outputs

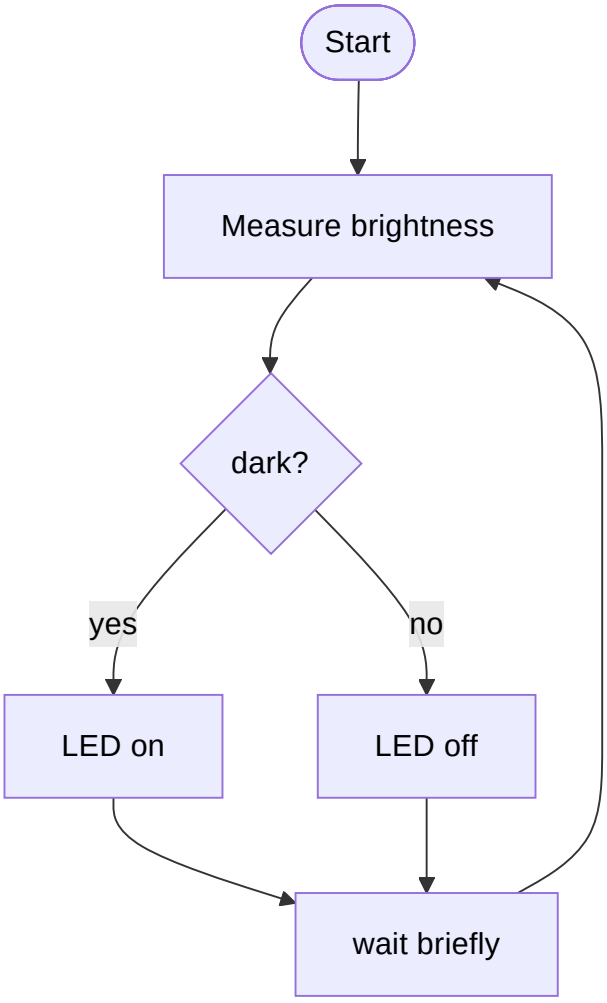
This is how you sketch your gadget in week 2:




```mermaid
flowchart LR
    B[Button] --> G((Night light))
    L[Light sensor] --> G
    G --> LED[LED]
```

Example 2: flowchart with a decision

With a **diamond** and labelled arrows you draw an `if`:




```mermaid
flowchart TD
    S([Start]) --> M[Measure brightness]
    M --> D{dark?}
    D -->|yes| ON[LED on]
    D -->|no| OFF[LED off]
    ON --> W[wait briefly]
    OFF --> W
    W --> M
```

The arrow from `W` back to `M` is the `while True` loop.

State diagram: `stateDiagram-v2`

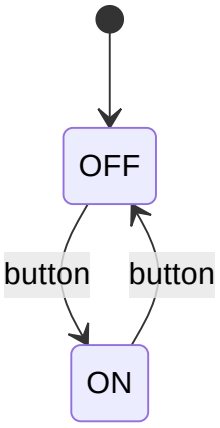
A state diagram shows which **states** a device can be in and which **event** takes it from one state to the next. More on this in week 4.

You write	Meaning
`[*] --> OFF`	the start state is OFF
`OFF --> ON: button`	transition from OFF to ON on the event “button”
`ON --> OFF: button / LED off`	event **/** what happens along the way

You write	Meaning
`OFF --> [*]`	end state (rarely needed)

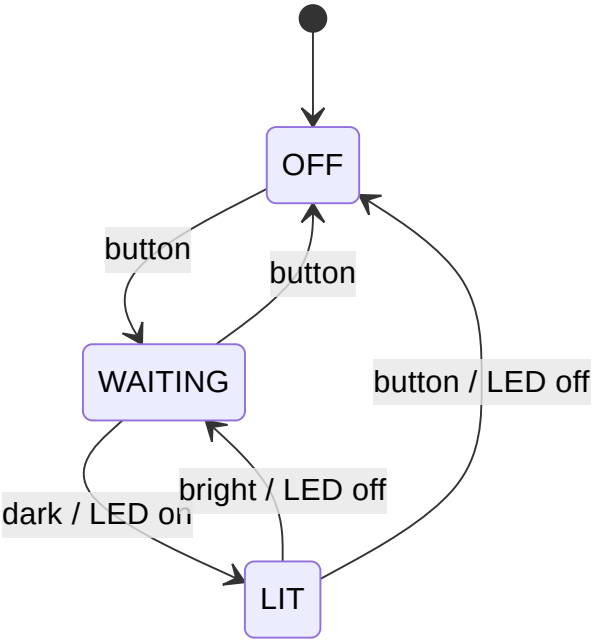
You don't have to declare states separately. As soon as a name appears in an arrow, it exists.

Example 3: the switch




````
```mermaid
stateDiagram-v2
    [*] --> OFF
    OFF --> ON: button
    ON --> OFF: button
```
````

Example 4: the night light



````
```mermaid
stateDiagram-v2
    [*] --> OFF
    OFF --> WAITING: button
    WAITING --> LIT: dark / LED on
    LIT --> WAITING: bright / LED off
    WAITING --> OFF: button
    LIT --> OFF: button / LED off
```
````

When the diagram doesn't appear

Mermaid is strict: a single typo, and instead of the picture you get an error message or nothing at all. Check in this order:

- Does the block start with ```` ```mermaid ```` (lower case) and end with ```` ``` ````?
- Is the first line exactly `flowchart LR`, `flowchart TD` or `stateDiagram-v2`?
- Are the **IDs** free of spaces and special characters? `LED_ON` works, `LED on` doesn't.
- Does a label contain brackets or quotes? Then put it in quotation marks: `A["Button (red)"]`.
- Is a flowchart node called `end`? That is a reserved word. Use `End` or `E([End])`.
- Is the arrow written correctly? It has **two** dashes: `-->`.

Paper first, then Mermaid

Work out the diagram with pen and paper first. Mermaid is for writing it up neatly, not for thinking. Once it works in mermaid.live, copy the block into your `.md` file.

More

- mermaid.live: online editor with instant preview
- Creating diagrams on GitHub
- Mermaid documentation with every diagram type